OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 5

ANALYSING DIGITAL LOGIC SYSTEMS

SUGGESTED TIME: 3 PERIODS

IN THIS CHAPTER

This chapter equips learners with the algebraic tools needed to analyse, simplify and compare digital logic systems. From Boolean identities to Karnaugh maps, students learn to minimise logic expressions and justify that a simplified circuit is equivalent to the original — a core skill for the GCE Advanced Level paper.

- By the end of this chapter I can state and apply the fundamental laws and theorems of Boolean algebra (commutative, associative, distributive, absorption, De Morgan's).
- By the end of this chapter I can prove the equivalence of two logic expressions using algebraic manipulation or truth tables.
- By the end of this chapter I can convert any logic expression into Sum-of-Products (SOP) and Product-of-Sums (POS) standard forms.
- By the end of this chapter I can write the canonical (minterm/maxterm) form of a function from its truth table.
- By the end of this chapter I can simplify a Boolean expression algebraically and verify the result.
- By the end of this chapter I can simplify functions of 2, 3 and 4 variables using Karnaugh maps, including don't-care conditions.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Boolean Algebra: Variables, Operations and Fundamental Laws
2. Algebraic Simplification and Standard Forms (SOP & POS)
3. Karnaugh Map Minimisation
4. Analysing and Designing Complete Logic Systems
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can state and apply the fundamental laws and theorems of Boolean algebra (commutative, associative, distributive, absorption, De Morgan's).
- By the end of this chapter I can prove the equivalence of two logic expressions using algebraic manipulation or truth tables.
- By the end of this chapter I can convert any logic expression into Sum-of-Products (SOP) and Product-of-Sums (POS) standard forms.
- By the end of this chapter I can write the canonical (minterm/maxterm) form of a function from its truth table.
- By the end of this chapter I can simplify a Boolean expression algebraically and verify the result.
- By the end of this chapter I can simplify functions of 2, 3 and 4 variables using Karnaugh maps, including don't-care conditions.
- By the end of this chapter I can implement a minimised expression using basic gates and using NAND-only or NOR-only gates.
- By the end of this chapter I can analyse a real-world control problem, model it with a truth table, minimise the logic and propose a circuit.

PREREQUISITES

- Logic gates (AND, OR, NOT, NAND, NOR, XOR) and their truth tables (Lower Sixth).

- Binary number system and binary variables taking values 0 and 1.
- Reading and constructing truth tables for combinational circuits.
- Deriving a logic expression from a simple circuit diagram.
- Basic algebraic manipulation (factorisation, expansion) from Ordinary Level mathematics.

1 BOOLEAN ALGEBRA: VARIABLES, OPERATIONS AND FUNDAMENTAL LAWS

Opening situation. In Douala, a water-tank controller must switch on the pump only when the tank is low and the ENEO supply is present, or when a manual override button is pressed. The technician's first circuit uses six gates where three would do. Every extra gate costs money, consumes power and adds a point of failure. How can we be sure two different circuits do exactly the same job? And how can we systematically find the simplest one? Boolean algebra, the mathematics of 0 and 1 invented by George Boole, gives us the tools to answer these questions rigorously — the same tools used to design the chips inside every phone in Cameroon.

1.1 Boolean Variables and Constants

DEFINITION **Boolean algebra and Boolean variable**

A **Boolean algebra** is a mathematical structure defined on a set containing exactly two elements, written 0 and 1, together with three operations: complement (NOT), logical sum (OR) and logical product (AND). A **Boolean variable** is a quantity that can take only one of the two values 0 or 1. The values 0 and 1 are called the **Boolean constants**.

In digital electronics, these two values correspond to physical voltage levels: 0 corresponds to the **LOW** level (near 0 V, meaning false, off, absent) and 1 corresponds to the **HIGH** level (near the supply voltage, e.g. 5 V, meaning true, on, present). In our pump example, we can define three input variables: $L = 1$ if the tank is low, $E = 1$ if ENEO power is present, $M = 1$ if the manual override is pressed; and one output $P = 1$ when the pump must run. The whole behaviour of the controller is then a Boolean equation.

WARNING **0 and 1 are not ordinary numbers**

In Boolean algebra, $1 + 1 = 1$, not 2. The symbols 0 and 1 are states, not quantities. Never apply the rules of ordinary arithmetic to Boolean expressions.

1.2 The Three Basic Operations

DEFINITION **Basic operations**

Let A and B be Boolean variables.

- The **complement** (NOT) of A, written A' (also $\bar{A}$ or $\neg A$), is 1 when $A = 0$ and 0 when $A = 1$.
- The **logical sum** (OR), written $A + B$, is 1 if at least one of A, B is 1.

- The **logical product** (AND), written $A \cdot B$ or simply AB, is 1 only if both A and B are 1.

Each operation is completely defined by its **truth table**, which lists the output for every possible combination of inputs:

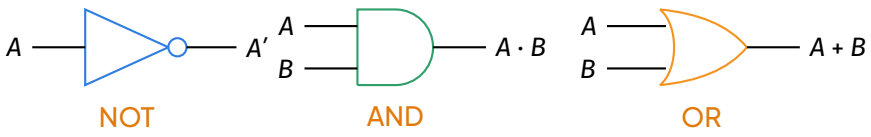
A	A'
0	1
1	0

A	B	$A + B$	$A \cdot B$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

KEY POINTS **Order of precedence**

When several operations appear in one expression, evaluate them in this order: parentheses first, then NOT, then AND, then OR. Thus $A + B \cdot C$ means $A + (B \cdot C)$, and AB' means $A \cdot (B')$, never $(AB)'$.

Each operation is realised physically by a **logic gate**. The figure below shows the standard symbols with their Boolean notation.



Gate symbols for NOT, AND and OR with their Boolean notation.

EXEMPLE **The pump controller as a Boolean expression**

The pump must run when (tank low AND ENEO present) OR (manual override). With L, E, M as defined above:

$$P = L \cdot E + M.$$

This single expression replaces any verbal description and can be analysed, proved and simplified using the laws below.

1.3 Postulates of Boolean Algebra

The **postulates** are the starting rules, accepted without proof, from which all theorems are derived. Each comes in a dual pair: one form built around OR and the constant 0, the other around AND and the constant 1.

PROPERTY **Fundamental postulates**

For any Boolean variable A:

- Identity elements: $A + 0 = A$ and $A \cdot 1 = A$.
- Domination: $A + 1 = 1$ and $A \cdot 0 = 0$.
- Idempotence: $A + A = A$ and $A \cdot A = A$.
- Complement laws: $A + A' = 1$ and $A \cdot A' = 0$.
- Involution: $(A')' = A$.

Each is verified by perfect induction, i.e. by checking the two cases $A = 0$ and $A = 1$ in a truth table.

EXEMPLE **Verifying domination by perfect induction**

For $A + 1 = 1$: if $A = 0$, then $0 + 1 = 1$; if $A = 1$, then $1 + 1 = 1$. The identity holds in all cases, so it is proved. Similarly $A \cdot 0 = 0$: $0 \cdot 0 = 0$ and $1 \cdot 0 = 0$.

1.4 Commutative, Associative and Distributive Laws

PROPERTY **Laws of combination**

For all Boolean variables A, B, C:

- Commutativity: $A + B = B + A$ and $A \cdot B = B \cdot A$.
- Associativity: $(A + B) + C = A + (B + C)$ and $(A \cdot B) \cdot C = A \cdot (B \cdot C)$.
- Distributivity of AND over OR: $A(B + C) = AB + AC$.
- Distributivity of OR over AND: $A + BC = (A + B)(A + C)$.

The first distributive law looks like ordinary algebra; the second does not, and it is specific to Boolean algebra. It is one of the most useful tools in simplification, so we prove it in full.

WORKED EXAMPLE 1 **Proving the second distributive law**

Prove that $A + BC = (A + B)(A + C)$.

SOLUTION

Expand the right-hand side using the first distributive law:

$$\begin{aligned}(A+B)(A+C) &= AA + AC + BA + BC \\ &= A + AC + AB + BC && \text{(idempotence } AA = A\text{)} \\ &= A(1 + C + B) + BC && \text{(factorising)} \\ &= A \cdot 1 + BC && \text{(domination } 1 + C + B = 1\text{)} \\ &= A + BC && \text{(identity } A \cdot 1 = A\text{).}\end{aligned}$$

Hence $A+BC = (A+B)(A+C)$. A truth-table check over the 8 combinations of (A, B, C) confirms the result.

WARNING **Trap: distributing as in ordinary algebra**

A very common mistake is to write $A + BC = (A + B)C$. This is false: take $A = 1$, $B = 0$, $C = 0$: the left side gives $1 + 0 = 1$ but the right side gives $(1 + 0) \cdot 0 = 0$. The correct second distributive law is $A+BC = (A+B)(A+C)$ — the A is duplicated and combined with each term of the product. Do not import habits from ordinary arithmetic.

1.5 Absorption and Redundancy Theorems

PROPERTY **Absorption and redundancy**

For all Boolean variables A, B:

- Absorption (sum form): $A + AB = A$.
- Absorption (product form): $A(A + B) = A$.
- Redundancy theorem: $A + A'B = A + B$, and dually $A(A' + B) = AB$.

Proofs (examinable). For absorption:

$$A + AB = A(1 + B) = A \cdot 1 = A.$$

$$A(A + B) = AA + AB = A + AB = A.$$

For redundancy, using the second distributive law:

$$A + A'B = (A + A')(A + B) = 1 \cdot (A + B) = A + B.$$

Intuitively, absorption says that if A alone already makes the expression true, adding "A and something" brings nothing new. Redundancy says that in $A + A'B$, the A' is useless: the term only matters when $A = 0$, in which case B alone decides.

EXEMPLE **Simplifying with absorption and redundancy**

Simplify $F = X + XY + X'Z$.

$$F = X + X'Z \quad \text{(absorption removes } XY\text{)} = X + Z \quad \text{(redundancy)}.$$

A three-gate circuit collapses to a single OR gate — exactly the kind of saving our Douala technician needs.

1.6 De Morgan's Theorems

PROPERTY **De Morgan's theorems**

For all Boolean variables A and B:

$$(A + B)' = A' \cdot B' \qquad \text{and} \qquad (A \cdot B)' = A' + B'.$$

Generalised to n variables: the complement of a sum is the product of the complements, and the complement of a product is the sum of the complements:

$$(X_1 + X_2 + \cdots + X_n)' = X_1' \cdot X_2' \cdots X_n', \qquad (X_1 X_2 \cdots X_n)' = X_1' + X_2' + \cdots + X_n'.$$

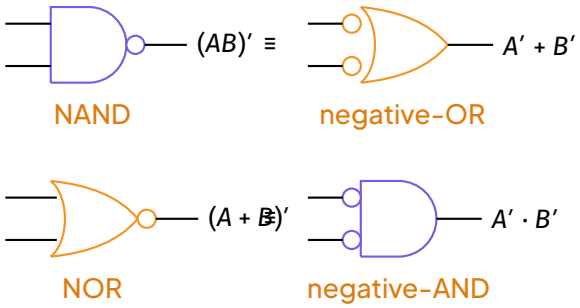
(Proof by truth table below; examinable.)

EXEMPLE **Proof of $(A + B)' = A' \cdot B'$ by perfect induction**

A	B	$A + B$	$(A + B)'$	A'	B'	$A' \cdot B'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

The columns $(A+B)'$ and $A' \cdot B'$ are identical for all four combinations, so the two expressions are equal. The same method proves $(AB)' = A' + B'$.

Graphically, De Morgan's theorems mean that pushing the inversion bubble through a gate changes AND into OR and vice versa. A NAND gate (AND with inverted output) is identical to an OR gate with inverted inputs ("negative-OR"); a NOR gate is identical to an AND gate with inverted inputs ("negative-AND").



Bubble pushing: De Morgan's theorem swaps the gate type and moves the inversion bubbles.

WARNING **$A'B'$ is NOT the complement of AB**

The complement of AB is $(AB)' = A' + B'$, not $A'B'$. Check with $A = 1$, $B = 0$: $(AB)' = (0)' = 1$, but $A'B' = 0 \cdot 1 = 0$. When complementing a product (or a sum), you must apply De Morgan: complement each variable and change the operation. Similarly $(A+B)' = A'B'$, never $A' + B'$.

1.7 The Principle of Duality

DEFINITION **Dual of an identity**

The **dual** of a Boolean identity is obtained by interchanging everywhere: $+$ with $\cdot$, and 0 with 1 (variables and complements stay unchanged).

PROPERTY **Principle of duality**

If a Boolean identity is true, its dual is also true. This is because every postulate comes in a dual pair, so any proof can be mirrored step by step.

EXEMPLE **Using duality**

From $A + AB = A$ (proved above), duality immediately gives $A(A + B) = A$ with no new proof. From $A + A'B = A + B$ we get $A(A' + B) = AB$.

KEY POINTS **Summary of postulates and laws in dual pairs**

Name	OR form	AND form (dual)
Identity	$A + 0 = A$	$A \cdot 1 = A$
Domination	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotence	$A + A = A$	$A \cdot A = A$
Complement	$A + A' = 1$	$A \cdot A' = 0$
Involution	$(A')' = A$	
Commutativity	$A + B = B + A$	$AB = BA$
Associativity	$(A + B) + C = A + (B + C)$	$(AB)C = A(BC)$
Distributivity	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption	$A + AB = A$	$A(A + B) = A$
Redundancy	$A + A'B = A + B$	$A(A' + B) = AB$
De Morgan	$(A + B)' = A' \cdot B'$	$(A \cdot B)' = A' + B'$

1.8 Proving Equivalence of Two Expressions

There are two great methods to prove that two Boolean expressions are equal.

METHOD **Verifying equivalence by perfect induction (truth table)**

1. List all 2^n combinations of the n input variables.
2. Evaluate the first expression for every combination, step by step (one column per sub-expression).
3. Evaluate the second expression in the same table.
4. Compare the two final columns: if they are identical in every row, the expressions are equivalent; a single differing row proves they are not.

The second method is **algebraic derivation**: starting from one expression, apply the postulates and theorems above until the other expression is obtained. This is faster for skilled students and is the basis of circuit minimisation, which we develop in the next section.

WORKED EXAMPLE 2 **Two circuits, one job**

The Douala technician's circuit implements $F = M + LE + LEM'$; a colleague proposes $G =$

$M + LE$. Show the two circuits do exactly the same job.

SOLUTION

Algebraic proof. Factor the last two terms of F:

$$F = M + LE(1 + M') = M + LE \cdot 1 = M + LE = G,$$

using domination ($1 + M' = 1$) and identity. So the term LEM' is useless and one AND gate can be removed. Truth-table check. With three variables there are $2^3 = 8$ rows:

L	E	M	LE	LEM'	$F = M + LE + LEM'$
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	0	0	0
1	0	1	0	0	1
1	1	0	1	0	1
1	1	1	1	0	1

The column of F equals M OR LE in every row, confirming $F = G$. The simpler circuit is cheaper, consumes less power and has fewer points of failure.

EXERCISE 1 **Practice**

1. Prove by perfect induction that $A(A + B) = A$.
2. Simplify algebraically, justifying each step: $F = AB + AB' + A'B$.
3. Using De Morgan, write the complement of $G = (A + B')(C + D)$ as a sum of products of complemented variables.
4. Give the dual of the identity $A + A'B = A + B$ and verify it for $A = 0$, $B = 1$.

SOLUTION **Exercise 1**

1. If $A = 0$: $0(0 + B) = 0$. If $A = 1$: $1(1 + B) = 1$. Both cases give A, so $A(A + B) = A$. 2. $F = A(B + B') + A'B = A \cdot 1 + A'B = A + A'B = A + B$ (complement law, identity, then redundancy). 3. $G' = (A + B')' + (C + D)' = A'B + C'D'$. 4. Dual: $A(A' + B) = AB$. For $A = 0$, $B = 1$: left side $0(1 + 1) = 0$; right side $0 \cdot 1 = 0$. Verified.

DID YOU KNOW? **George Boole and Claude Shannon**

George Boole published his "algebra of logic" in 1854 as pure mathematics, with no machine in mind. Almost a century later, Claude Shannon showed that Boolean algebra describes exactly the behaviour of switching circuits — giving birth to digital electronics. Every processor in every phone used in Cameroon today is, at its heart, billions of tiny switches obeying the laws you have just learned.

2 ALGEBRAIC SIMPLIFICATION AND STANDARD FORMS (SOP & POS)

In the previous section we established the vocabulary and the fundamental laws of Boolean algebra: variables, the three basic operations (NOT, AND, OR), and the identities such as $X+1 = 1$, $X \cdot X' = 0$, absorption and De Morgan's theorems. Those laws are not just abstract curiosities. They are the tools that allow us to transform a logic expression into an equivalent one that is cheaper to build. This section shows you how to use them strategically, and introduces the two standard forms — SOP and POS — in which any Boolean function can be written, together with the canonical (minterm/maxterm) forms that come directly from a truth table.

2.1 Why Simplify a Boolean Expression?

Imagine a technician in Douala who must build the control circuit for a water pump. The specification gives the expression

$$F = AB + AB' + A'BC.$$

Built as written, this needs three AND gates (one with 3 inputs), one 3-input OR gate and two inverters. But a few lines of algebra give

$$F = A(B + B') + A'BC = A + A'BC = A + BC,$$

which needs only **one** AND gate and **one** OR gate — and no inverter at all. The two circuits produce exactly the same outputs for every input combination, yet the second one is:

- **cheaper**: fewer integrated circuits to buy;
- **lower power**: fewer gates switching means less energy consumed and less heat;
- **more reliable**: every gate and every solder joint is a potential point of failure; fewer components means fewer failures;
- **faster**: fewer gate levels between input and output means a shorter propagation delay.

KEY POINTS **The goal of simplification**

Two expressions are **equivalent** if they have the same truth table. Simplification means

finding an equivalent expression with the fewest possible literals and terms, because each literal corresponds to a gate input and each term to a gate.

2.2 Strategy of Algebraic Simplification

There is no automatic recipe, but experienced designers repeatedly use the same four moves:

1. **Expand** (distribute) to expose common factors: $X(Y + Z) = XY + XZ$.
2. **Factorise** to apply $X + X' = 1$: $AB + AB' = A(B + B') = A$.
3. **Apply absorption laws**: $X + XY = X$ and $X + X'Y = X + Y$.
4. **Eliminate redundant terms** with the consensus theorem (below).

EXEMPLE **Step-by-step simplification**

Simplify $F = A'B + A'BC + AB'$.

$$
\begin{aligned}
F &= A'B + A'BC + AB' \\
&= A'B(1 + C) + AB' && \text{(factorise } A'B\text{)} \\
&= A'B + AB' && (1 + C = 1) \\
&\text{no further reduction: } F = A'B + AB'.
\end{aligned}
$$

Check with $A = 0, B = 1, C = 0$: original gives $1 + 0 + 0 = 1$; simplified gives $1 + 0 = 1$. ✓

EXEMPLE **Using** $X + X'Y = X + Y$

Simplify $F = A + A'B$.

$$F = A + A'B = (A + A')(A + B) = 1 \cdot (A + B) = A + B.$$

One OR gate replaces an inverter, an AND gate and an OR gate.

2.3 The Consensus Theorem

Some terms in an expression are redundant: they add no new 1s to the truth table. The consensus theorem identifies them.

PROPERTY **Consensus theorem — proof examinable**

$$AB + A'C + BC = AB + A'C.$$

The term BC is called the **consensus term** of AB and $A'C$; it is redundant. Dual form: $(A + B)(A' + C)(B + C) = (A + B)(A' + C)$.

Proof. Starting from the left-hand side:

$$\begin{aligned} AB + A'C + BC &= AB + A'C + BC(A + A') && (A + A' = 1) \\ &= AB + A'C + ABC + A'BC && \text{(distribution)} \\ &= AB(1 + C) + A'C(1 + B) && \text{(factorisation)} \\ &= AB + A'C && (1 + X = 1). \ \blacksquare \end{aligned}$$

EXEMPLE **Applying consensus**

Simplify $F = AB + A'C + BC$. Here AB contains A and $A'C$ contains A'; the consensus of these two terms is BC, which is present. Therefore

$$F = AB + A'C.$$

A whole AND gate and one input of the OR gate are saved.

WARNING **Do not remove the wrong term**

Consensus removes the term that contains neither the complemented nor the uncomplemented variable of the other two terms. In $AB + A'C + BC$ you may remove BC, never AB or $A'C$. When in doubt, verify with a truth table.

2.4 Literals, Product Terms, Sum Terms: SOP and POS

DEFINITION **Literal, product term, sum term**

A **literal** is a Boolean variable or its complement: A, B', C are literals.
A **product term** is a single literal or an AND of literals: $AB'C$.
A **sum term** is a single literal or an OR of literals: $A + B' + C$.

DEFINITION **SOP and POS**

An expression is in **SOP** (Sum Of Products) form if it is an OR of one or more product terms:

$$F = AB + A'C + B'.$$

It is in **POS** (Product Of Sums) form if it is an AND of one or more sum terms:

$$F = (A + B)(A' + C)(B' + C).$$

An expression such as $(AB + C)(A + D)$ is neither SOP nor POS.

SOP expressions map naturally onto a **two-level AND–OR circuit**: a first level of AND gates (one per product term) feeding a single OR gate. POS expressions map onto an OR–AND circuit. Two levels means a short propagation delay, which is why these forms dominate practical design.

2.5 Minterms and Maxterms

DEFINITION **Minterm and maxterm**

For n variables, a **minterm** m_i is a product term in which every variable appears exactly once, complemented or not. A minterm equals 1 for exactly one row of the truth table.
A **maxterm** M_i is a sum term in which every variable appears exactly once. A maxterm equals 0 for exactly one row.
Numbering convention: with the variables in a declared order (e.g. A, B, C with A most significant), read the row as a binary number where a plain variable is 1 and a complemented variable is 0; the decimal value is the index i. Note that $M_i = m_i'$ (De Morgan).

A	B	C	Row i	Minterm m_i	Maxterm M_i
0	0	0	0	$A'B'C'$	$A + B + C$
0	0	1	1	$A'B'C$	$A + B + C'$
0	1	0	2	$A'BC'$	$A + B' + C$
0	1	1	3	$A'BC$	$A + B' + C'$
1	0	0	4	$AB'C'$	$A' + B + C$
1	0	1	5	$AB'C$	$A' + B + C'$
1	1	0	6	ABC'	$A' + B' + C$
1	1	1	7	ABC	$A' + B' + C'$

Minterms and maxterms for three variables A, B, C (A most significant).

For two variables A, B the minterms are $m_0 = A'B'$, $m_1 = A'B$, $m_2 = AB'$, $m_3 = AB$. With n variables there are exactly 2^n minterms and 2^n maxterms.

WARNING **Variable order matters!**

The numbering m_i is only meaningful if the order of the variables is stated. With order (A, B, C), $A'BC$ is m_3 (binary 011); with order (C, B, A) the same product would be m_6. In the GCE, always write the variable order before using Σm or ΠM notation, and use the order given in the question.

2.6 Canonical SOP: Sum of Minterms

DEFINITION **Canonical SOP**

The **canonical SOP** of a function is the OR of the minterms of all rows where $F = 1$. Every Boolean function has a unique canonical SOP. We write

$$F(A, B, C) = \Sigma m(1, 3, 6)$$

to mean $F = m_1 + m_3 + m_6 = A'B'C + A'BC + ABC'$.

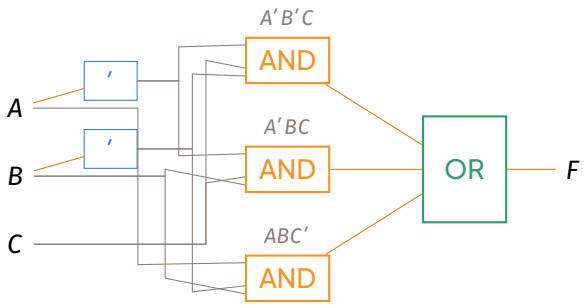
METHOD **From truth table to canonical SOP**

1. Locate every row of the truth table where $F = 1$.
2. For each such row, write the minterm: variable plain if it is 1 in that row, complemented if it is 0.
3. OR all the minterms together.
4. Optionally compress with Sigma notation: $F = \Sigma m$(row numbers).

EXEMPLE **Canonical SOP from a truth table**

A function is 1 for $(A, B, C) = (0, 0, 1), (0, 1, 1)$ and $(1, 1, 0)$. Then

$$F = A'B'C + A'BC + ABC' = \Sigma m(1, 3, 6).$$



Two-level AND–OR implementation of the canonical SOP $F = A'B'C + A'BC + ABC'$.

2.7 Canonical POS: Product of Maxterms

DEFINITION **Canonical POS**

The **canonical POS** of a function is the AND of the maxterms of all rows where $F = 0$. We write

$$F(A, B, C) = \Pi M(0, 2, 4, 5, 7).$$

For each zero row, write the sum term with the variable plain if it is 0 and complemented if it is 1 (the opposite convention to minterms), then AND the terms.

PROPERTY **Link between the two canonical forms**

The minterm list and the maxterm list of the same function are complementary: together they contain every index from 0 to $2^n - 1$, with no overlap. Hence

$$F = \Sigma m(1, 3, 6) = \Pi M(0, 2, 4, 5, 7),$$

and the complement of F is obtained by swapping the lists:

$$F' = \Sigma m(0, 2, 4, 5, 7) = \Pi M(1, 3, 6).$$

EXEMPLE **Converting SOP to POS and complementing**

Given $F(A, B, C) = \Sigma m(1, 3, 6)$:

- Canonical POS: $F = \Pi M(0, 2, 4, 5, 7) = (A + B + C)(A + B' + C)(A' + B + C)(A' + B + C')(A' + B' + C')$.
- Complement: $F' = \Sigma m(0, 2, 4, 5, 7)$.

2.8 Converting a Non-Standard Expression to Canonical Form

METHOD **Expanding with $(X + X')$**

To write any SOP expression as a sum of minterms:

1. In each product term, identify the missing variable(s).
2. Multiply the term by $(X + X')$ for each missing variable X.
3. Distribute, then remove any duplicate minterms (since $m_i + m_i = m_i$).

For POS, add XX' inside a sum term and use $A + BC = (A + B)(A + C)$.

WORKED EXAMPLE 3 **Canonical form of $F = AB + A'C$**

Write $F(A, B, C) = AB + A'C$ in canonical SOP.

SOLUTION

The term AB misses C; the term $A'C$ misses B.

$$\begin{aligned} F &= AB(C + C') + A'C(B + B') \\ &= ABC + ABC' + A'BC + A'B'C \\ &= m_7 + m_6 + m_3 + m_1 = \Sigma m(1.3, 6.7). \end{aligned}$$

2.9 GCE Exam Extra: NAND-Only and NOR-Only Implementations

NAND and NOR gates are called universal because any function can be built from either type alone — and in practice they are the cheapest gates to manufacture. The GCE frequently asks for a NAND-only (or NOR-only) circuit.

METHOD **NAND-only for SOP, NOR-only for POS**

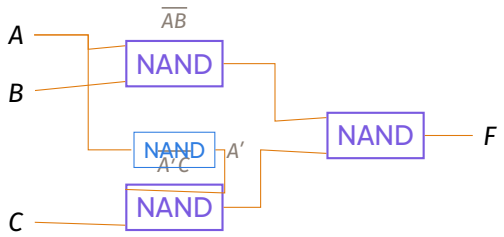
SOP → NAND-only: complement the whole expression twice (double bar), then apply De Morgan to the lower bar:

$$F = AB + A'C = \overline{\overline{AB + A'C}} = \overline{\overline{AB} \cdot \overline{A'C}}.$$

Each $\overline{AB}$ is a NAND; the outer gate is again a NAND. Rule: replace every AND and OR of the two-level circuit by NAND.
POS → NOR-only: same idea on a POS expression:

$$F = (A + B)(A' + C) = \overline{\overline{(A + B)(A' + C)}} = \overline{\overline{A + B} + \overline{A' + C}},$$

so every gate becomes a NOR. A single input is inverted by tying the two inputs of a NAND (or NOR) together.



NAND-only implementation of $F = AB + A'C$ via double complementation; the inverter is a NAND with tied inputs.

WORKED EXAMPLE 4 **Full worked problem: from truth table to NAND circuit**

A security system has three sensors A, B, C. The alarm F must ring ($F = 1$) for the input combinations 1, 3, 6, 7. Give the canonical SOP, simplify it, and draw the NAND-only form.

SOLUTION

Canonical SOP: $F = \Sigma m(1.3, 6.7) = A'B'C + A'BC + ABC' + ABC$.
Simplification:

$$F = A'C(B' + B) + AB(C' + C) \qquad \text{(grouping)}$$
$$= A'C + AB.$$

NAND-only: $F = \overline{\overline{A'C} \cdot \overline{AB}}$ — two NAND gates feeding a third NAND, with A' produced by a NAND with tied inputs (the circuit of the figure above).

EXERCISE 2 **Practice**

1. Simplify algebraically, justifying each step: $F = AB + A'B + AB'C$.
2. Write $F(A, B, C) = A + B'C$ in canonical SOP (Sigma notation).
3. Given $F(A, B, C) = \Sigma m(0, 2, 5)$, write the canonical POS and the minterm list of F'.
4. Implement $F = (A + B')(A' + C)$ using NOR gates only.

SOLUTION **Exercise 1**

1. $F = B(A + A') + AB'C = B + AB'C = (B + A)(B + B'C) = (A + B)(B + C)$, or in SOP: $B + AC$ using $B + AB'C = B + AC$ (rule $X + X'Y = X + Y$ with $X = B$).
2. $A = A(B + B')(C + C') = ABC + ABC' + AB'C + AB'C'$ (minterms 4, 5, 6, 7); $B'C = B'C(A + A') = AB'C + A'B'C$ (minterms 1, 5). Removing the duplicate m_5: $F = \Sigma m(1, 4, 5, 6, 7)$.
3. $F = \Pi M(1, 3, 4, 6, 7)$; $F' = \Sigma m(1, 3, 4, 6, 7)$.
4. $F = \overline{\overline{A + B'} + \overline{A' + C}}$: two NOR gates (with B' and A' from NORs with tied inputs) feeding a third NOR.

KEY POINTS **Key points of this section**

- Simplification reduces gates, cost, power and failure points; the main tools are factorisation, absorption and consensus.
- $AB + A'C + BC = AB + A'C$: the consensus term is redundant.
- SOP = OR of product terms; POS = AND of sum terms.
- Canonical SOP lists the minterms of the 1-rows: $F = \Sigma m(\ldots)$; canonical POS lists the maxterms of the 0-rows: $F = \Pi M(\ldots)$; the two lists are complementary.
- Any expression becomes canonical by multiplying by $(X + X')$ for each missing variable.
- SOP ⇒ NAND-only and POS ⇒ NOR-only by double complementation and De Morgan.

In the next section we shall see that algebraic simplification becomes difficult with four or more variables, and we shall replace it by a powerful graphical method: the Karnaugh map.

3 KARNAUGH MAP MINIMISATION

In the previous section we learned how to simplify Boolean expressions algebraically, by applying the laws of Boolean algebra step by step. That method works, but it has a serious weakness: there is no systematic recipe telling us which law to apply next, and we can never be absolutely sure that the expression we end up with is truly the simplest possible one. In this section we study a **graphical** method — the **Karnaugh map** (or **K-map**) — invented by Maurice Karnaugh at Bell Labs in 1953, which turns minimisation into a visual pattern-recognition exercise and guarantees a minimal sum-of-products (SOP) form for functions of up to four variables (and with care, five or six).

3.1 Why We Need a Graphical Method

Consider simplifying $F(A, B, C) = \bar{A}BC + AB\bar{C} + ABC + \bar{A}B\bar{C}$. Algebraically, a skilful student might spot that $AB\bar{C} + ABC = AB$ and that $\bar{A}BC + \bar{A}B\bar{C} = \bar{A}B$, giving $F = AB + \bar{A}B = B$. But nothing in the algebraic method told us to group the terms that way; it required intuition and luck. Worse, if we had stopped one step earlier we might believe $AB + \bar{A}B$ was the final answer when B was possible.

WARNING **Limits of algebraic simplification**

Algebraic simplification has two major drawbacks:

- It requires intuition: there is no systematic procedure, so two students may obtain two different "simplified" forms.
- It gives no guarantee of minimality: you can never be sure that no simpler equivalent expression exists.

The Karnaugh map removes both problems: it displays the function so that all possible simplifications become visible as geometric groupings.

The key idea is simple. Two minterms can be combined (simplified) exactly when they differ in **one single variable**, because then $X \cdot v + X \cdot \bar{v} = X$. If we could draw the truth table so that physically adjacent cells always differ in exactly one variable, then every simplification would appear as a block of adjacent 1s. That is precisely what a K-map does, using **Gray code** ordering.

DID YOU KNOW? **Maurice Karnaugh and the Bell Labs legacy**

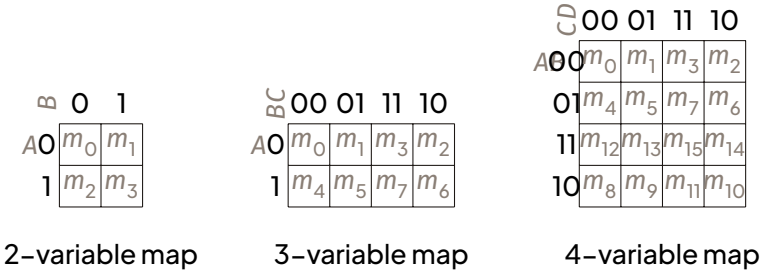
Maurice Karnaugh (1924–2022) developed the map while working at Bell Telephone Laboratories to simplify telephone switching circuits. His 1953 paper "The Map Method for Synthesis of Combinational Logic Circuits" gave engineers a practical tool that replaced tedious Boolean algebra. The method is essentially a two-dimensional truth table arranged in reflected binary code (Gray code), named after Frank Gray who patented it for pulse-code modulation in 1947. In Cameroon's telecommunications infrastructure, the same principles underlie the digital multiplexers that carry thousands of voice channels over fibre-optic links.

3.2 Structure of Karnaugh Maps and Gray-Code Ordering

A K-map is a grid with one cell per minterm of the function: 2^n cells for n variables. The rows and columns are labelled in **Gray code** (00, 01, 11, 10) so that consecutive labels differ in exactly one bit.

DEFINITION **Adjacency on a K-map**

Two cells are **adjacent** if their labels differ in exactly one variable. Adjacent cells are next to each other horizontally or vertically, including across the borders: the leftmost and rightmost columns are adjacent, and the top and bottom rows are adjacent (the map "wraps around" like a torus in both directions).



2-variable map 3-variable map 4-variable map

Karnaugh maps for 2, 3 and 4 variables. Note the Gray-code order 00, 01, 11, 10 on both axes and the resulting minterm positions (e.g. m_3 and m_2 are swapped relative to numeric order).

WARNING **The map wraps around**

Because of the Gray-code labelling, the first and last columns are adjacent, and the top and bottom rows are adjacent. In the 4-variable map, m_0 is adjacent to m_2 (same row, column wrap-around) and to m_8 (same column, row wrap-around). The four corners $\{m_0, m_2, m_8, m_{10}\}$ form a valid group of four. Forgetting wrap-around adjacency is one of the most frequent causes of non-minimal answers at the GCE.

3.3 Plotting a Function on the K-map

To plot a function, place a **1** in each cell whose minterm belongs to the function (from the truth table or from the minterm list $\sum m(\ldots)$), and a **0** elsewhere. If the function is given as a maxterm list $\prod M(\ldots)$, place 0s in those cells and 1s elsewhere.

EXEMPLE **Plotting from a minterm list**

Plot $F(A,B,C) = \sum m(1,2,3,6,7)$ on a 3-variable map (rows A, columns BC): cells m_1, m_2, m_3, m_6, m_7 receive a 1; cells m_0, m_4, m_5 receive a 0.

EXEMPLE **Plotting from a truth table**

Given the truth table for $F(A, B, C, D)$, transfer each output value to the corresponding cell of the 4-variable map using the row/column labels (AB for rows, CD for columns) as the input combination.

3.4 Grouping Rules and Reading Groups

PROPERTY **Size of groups**

On a K-map, only rectangular groups of 1, 2, 4, 8 or 16 adjacent cells containing 1s may be formed — always a power of two. A group of 2^k cells eliminates exactly k variables from the product term. (Justification: each doubling of the group pairs cells that differ in one further variable, which is then eliminated by $Xv + X\bar{v} = X$. The justification is instructive but not required as a formal proof at the GCE.)

WARNING **Only powers of two**

Groups of 3, 5, 6, 7 cells are forbidden, even if they look like a neat rectangle or L-shape. An L-shape of 3 cells must be split into overlapping groups of 2 and 2 (or 2 and 1). Groups may overlap — a cell containing 1 may belong to several groups. Overlapping is encouraged if it allows larger groups.

Reading a group. Within a group, look at each variable:

- if the variable **changes value** inside the group, it is **eliminated**;
- if it is **constant at 1**, it appears **uncomplemented** in the product;
- if it is **constant at 0**, it appears **complemented**.

DEFINITION **Implicant, prime implicant, essential prime implicant**

Let F be a Boolean function.

- An **implicant** of F is any product term whose 1-cells on the K-map are all 1-cells of F (any valid group).
- A **prime implicant** is an implicant that cannot be enlarged: it is a group that is not fully contained in any larger valid group.
- An **essential prime implicant** is a prime implicant that covers at least one 1-cell covered by no other prime implicant. Every essential prime implicant must appear in the

minimal SOP.

METHOD **K-map minimisation procedure (SOP)**

1. Plot: draw the K-map of the right size and enter the 1s (and X for don't-cares) from the truth table or minterm list.
2. Identify essential prime implicants: find 1-cells that can be covered in only one maximal way; loop those groups first.
3. Cover remaining 1s: loop the uncovered 1s into the fewest possible groups, each as large as possible (powers of two only, overlaps allowed, wrap-around allowed).
4. Read: for each group, write the product of the constant variables (eliminate the changing ones).
5. Write: the minimal SOP is the OR (sum) of all the group products. Check that every 1 is covered.
6. Verify: test the simplified expression against a few rows of the original truth table, especially rows where the original function is 0.

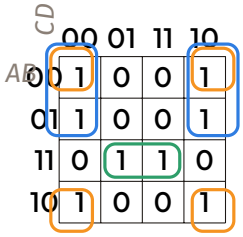
WORKED EXAMPLE 5 **Minimising a 4-variable function with wrap-around groups**

Minimise $F(A, B, C, D) = \sum m(0, 2, 4, 6, 8, 10, 13, 15)$.

SOLUTION

Step 1: Plot. Enter 1s at the listed minterms on a 4-variable map (AB rows, CD columns).
Step 2: Group.

- The 1s at m_0, m_2, m_8, m_{10} occupy the four corners of the map — a wrap-around group of 4 (left-right and top-bottom adjacency). This group is essential for m_0, m_2, m_8, m_{10}.
- The 1s at m_4, m_6 (row $AB = 01$, cols $CD = 00, 10$) are adjacent to m_0, m_2 (row 00) vertically. Together they form a group $\{m_0, m_2, m_4, m_6\}$ of 4 using column wrap-around (cols 00 and 10). This group is essential for m_4, m_6.
- The 1s at m_{13}, m_{15} (row $AB = 11$, cols $CD = 01, 11$) form a horizontal pair.



Orange: corner group of 4 ($\bar{B}\bar{D}$). Blue: column wrap group of 4 ($\bar{A}\bar{D}$). Green: pair (ABD).

Step 3: Read the groups.

- Corners $\{m_0, m_2, m_8, m_{10}\}$: A changes (0/1), C changes (0/1); $B = 0$, $D = 0$ constant $\Rightarrow \bar{B}\bar{D}$.
- Column group $\{m_0, m_2, m_4, m_6\}$: B changes (0/1), C changes (0/1); $A = 0$, $D = 0 \Rightarrow \bar{A}\bar{D}$.
- Pair $\{m_{13}, m_{15}\}$: $A = 1$, $B = 1$, $D = 1$; C changes $\Rightarrow ABD$.

Step 4: Write SOP. $F = \bar{B}\bar{D} + \bar{A}\bar{D} + ABD$.
Step 5: Verify.

- $m_{13}(1101)$: $\bar{B}\bar{D} = 0, \bar{A}\bar{D} = 0, ABD = 1 \Rightarrow F = 1$ ✓
- $m_1(0001)$ (a 0-cell): $\bar{B}\bar{D} = 0, \bar{A}\bar{D} = 0, ABD = 0 \Rightarrow F = 0$ ✓
- $m_6(0110)$: $\bar{B}\bar{D} = 1 \Rightarrow F = 1$ ✓

3.5 Don't-Care Conditions

In many real systems, some input combinations can never occur. For example, a circuit receiving a **BCD digit** (4 bits representing 0–9) will never see the codes 1010 to 1111 (10–15). The output for those combinations is irrelevant: we mark it **X** (**don't care**).

WARNING **A don't-care is not a 1**

An X cell does not have to be covered. Include an X in a group only if it makes the group larger (hence the term simpler). Never form a group containing only X cells — that would add a useless term.

EXEMPLE **Using don't-cares: Odd BCD digit detector**

A circuit must output 1 for BCD digits 1, 3, 5, 7, 9 (odd digits), with inputs $ABCD$ (A = most significant bit) and codes 10–15 don't-care. The 1s are m_1, m_3, m_5, m_7, m_9; the X cells are

m_{10}–m_{15}.

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	X	X	X	X
10	0	1	X	X

Odd-BCD-digit detector: the don't-care cells m_{10}–m_{15} let the five 1s merge into one group of 8, giving $F = D$.

The group of 8 (columns CD = 01 and 11, all four rows) keeps only D constant at 1, so $F = D$ — a dramatic simplification made possible only by the don't-cares. Without them, the best grouping would give $F = \bar{A}D + \bar{B}\bar{C}D$.

3.6 Obtaining a Minimal POS (Exam-Useful Extra)

The same map yields a minimal **product-of-sums (POS)**: group the **0-cells** exactly as before; this produces a minimal SOP for $\bar{F}$; then **complement the whole expression** and apply De Morgan's laws to obtain F in POS form.

EXEMPLE **POS from the 0s**

If grouping the 0s of a map gives $\bar{F} = \bar{A}B + C\bar{D}$, then

$$F = \overline{\bar{A}B + C\bar{D}} = \overline{(\bar{A}B)} \cdot \overline{(C\bar{D})} = (A + \bar{B})(\bar{C} + D),$$

a minimal POS with two sum terms.

METHOD **Minimal POS via K-map**

1. Plot the function (1s and 0s).
2. Group the 0-cells into largest possible power-of-two rectangles (using X cells if helpful).
3. Write the SOP expression for $\bar{F}$ from these groups.
4. Apply De Morgan's theorem: complement the whole expression and change each product to a sum, complementing each literal.

5. The result is a minimal POS for F.

3.7 Verification and Common Traps

METHOD **Verifying a minimised expression**

1. Pick each 1-cell of the original truth table and check the minimised SOP outputs 1 (at least one product term covers it).
2. Pick several 0-cells and check every product term outputs 0 there.
3. Equivalently, plot the K-map of your answer and compare it cell by cell with the original map.

WARNING **Frequent errors at the GCE**

- Labelling rows/columns in binary order (00, 01, 10, 11) instead of Gray code — the whole method collapses.
- Forgetting wrap-around adjacency (edges and the four corners).
- Making groups of 3 or 6, or diagonal (non-rectangular) groups.
- Covering don't-care cells that do not enlarge any group.
- Leaving a 1-cell uncovered, or choosing small groups when larger ones exist (non-minimal answer).
- Writing the POS directly by grouping 1s and reading sums — this is wrong. Group 0s for POS.

EXPERIMENT **Guided design: 7-segment decoder segment 'a'**

A 7-segment display shows digits 0–9. Segment 'a' (top horizontal) is ON for digits 0,2,3,5,6,7,8,9 and OFF for 1,4. Codes 10–15 are don't-care.

1. Draw the 4-variable K-map for segment 'a' (inputs A, B, C, D = BCD digit).
2. Find the minimal SOP expression.
3. Find the minimal POS expression.
4. Which form uses fewer gates if implemented with AND-OR vs OR-AND logic?

Hint: The SOP simplifies to $\bar{A} + \bar{B}C + \bar{B}D + B\bar{C}\bar{D}$ (or similar minimal form). The POS is often simpler for this function.

EXERCISE 3 **Practice**

1. Minimise $F(A, B, C) = \sum m(0, 2, 4, 5, 6)$ using a 3-variable K-map.
2. Minimise $G(A, B, C, D) = \sum m(1, 3, 5, 7, 9, 11, 13, 15)$. What do you notice about the result?
3. A BCD-based circuit outputs 1 for digits 0, 2, 4, 6, 8 (even digits) and is don't-care for 10–15. Find the minimal SOP.
4. For $H(A, B, C, D) = \sum m(0, 1, 2, 5, 8, 9, 10) + \sum d(4, 11, 14)$, find both minimal SOP and minimal POS.

SOLUTION **Exercise 1**

1. Plot m_0, m_2, m_4, m_5, m_6 (rows A, columns BC). Groups: $\{m_0, m_2, m_4, m_6\}$ (columns 00 and 10, both rows, wrap-around) gives $\bar{C}$; $\{m_4, m_5\}$ gives $A\bar{B}$. Hence $F = \bar{C} + A\bar{B}$. Check $m_6 = 110$: $\bar{C} = 1$ ✓; $m_1 = 001$: $\bar{C} = 0, A\bar{B} = 0$ ✓.
2. All 1s have $D = 1$: one group of 8, so $G = D$. The function depends on a single variable.
3. 1s at m_0, m_2, m_4, m_6, m_8; X at m_{10}–m_{15}. Using the X cells, columns $CD = 00$ and 10 (all rows, wrap-around) form a group of 8 with $D = 0$ constant: $F = \bar{D}$ (even-digit detector).
4. SOP: Plot 1s and Xs. Essential prime implicants: $\bar{B}\bar{D}$ (covers m_0, m_2, m_8, m_{10}), $\bar{A}\bar{C}$ (covers m_0, m_1, m_8, m_9), $A\bar{B}\bar{C}$ (covers m_8, m_9? wait). Let's trace carefully. Map 1s: 0,1,2,5,8,9,10. X: 4,11,14. Groups:

- Corners 0,2,8,10 + X4? No, X4 is row 01 col 00. Group $\{0, 2, 8, 10\} \rightarrow \bar{B}\bar{D}$.
- $\{0, 1, 8, 9\}$ (cols 00,01 rows 00,10) $\rightarrow \bar{B}\bar{C}$.
- $\{2, 10\}$ already covered. $\{5\}$ (0101) needs cover. Can pair with 4 (X) $\rightarrow \{4, 5\} = A\bar{B}\bar{C}$? Row 01 ($A = 0, B = 1$), Col 01 ($C = 0, D = 1$) $\rightarrow m_5$. Col 00 ($C = 0, D = 0$) $\rightarrow m_4$ (X). Group $\{4, 5\} \rightarrow \bar{A}B\bar{C}$.
- $\{11\}$ (X) adjacent to 9,10? 11 is 1011. Adjacent to 9(1001) D changes, 10(1010) C changes, 3(0011) A changes, 15(1111) B changes.
- $\{14\}$ (X) 1110.

Minimal SOP: $\bar{B}\bar{D} + \bar{B}\bar{C} + \bar{A}B\bar{C}$. POS: Group 0s (3,6,7,12,13,15) + X(4,11,14). 0s: $m_3(0011), m_6(0110), m_7(0111), m_{12}(1100), m_{13}(1101), m_{15}(1111)$. Xs: $m_4(0100), m_{11}(1011), m_{14}(1110)$. Group 0s:

- $\{3, 7, 11(X), 15\} \rightarrow$ col 11 ($C = 1, D = 1$) rows 00,01,10,11? Row 10 is $m_{10} = 1$. Row 01 $m_7 =$

$0, m_3 = 0$. Row 11 $m_{15} = 0, m_{11} = X$. Group $\{3, 7, 11, 15\} \rightarrow CD$.

- $\{6, 14(X)\}$ → row 01 col 10, row 11 col 10 → $B\bar{D}$? Row 01 ($A = 0, B = 1$), Row 11 ($A = 1, B = 1$) → $B = 1$. Col 10 ($C = 1, D = 0$) → $\bar{D}$. Term $B\bar{D}$.
- $\{12, 13, 14(X), 15\}$ → row 11 ($A = 1, B = 1$) all cols → AB.
- $\{4(X), 6\}$ → row 01 col 00, col 10 → $\bar{A}B\bar{C}$? Col 00 ($C = 0$), Col 10 ($C = 1$) → C changes. Row 01 ($A = 0, B = 1$). Term $\bar{A}B$.

$\bar{F} = CD + B\bar{D} + AB + \bar{A}B = CD + B(\bar{D} + A + \bar{A}) = CD + B$. $F = \overline{CD + B\bar{D} + B} = \overline{CD + B} = (\bar{C} + \bar{D})\bar{B} = \bar{B}\bar{C} + \bar{B}\bar{D}$. Check SOP: $\bar{B}\bar{D} + \bar{B}\bar{C} + \bar{A}B\bar{C}$. If $B = 0$, first two terms cover all. If $B = 1$, need $\bar{A}\bar{C}$. Matches POS $\bar{B}(\bar{C} + \bar{D}) = \bar{B}\bar{C} + \bar{B}\bar{D}$. Wait, SOP has extra term $\bar{A}B\bar{C}$. Check m_5 (0101): $\bar{B}\bar{D} = 0, \bar{B}\bar{C} = 0, \bar{A}B\bar{C} = 1(1)(1) = 1$. POS: $\bar{B}\bar{C} + \bar{B}\bar{D} = 0$. Mismatch! m_5 is a 1-cell. POS grouping missed covering $m_5 = 0$? m_5 is 1. 0s are 3,6,7,12,13,15. Group $\{3, 7, 11, 15\} \rightarrow CD$. Group $\{6, 14\} \rightarrow B\bar{D}$. Group $\{12, 13, 14, 15\} \rightarrow AB$. Group $\{4, 6\} \rightarrow \bar{A}B$. $\bar{F} = CD + B\bar{D} + AB + \bar{A}B = CD + B(\bar{D} + A + \bar{A}) = CD + B$. $F = \overline{CD + B} = (\bar{C} + \bar{D})\bar{B} = \bar{B}\bar{C} + \bar{B}\bar{D}$. Check m_5 (0101): $\bar{B}\bar{C} = 0, \bar{B}\bar{D} = 0$. $F = 0$. But m_5 is a 1-cell! Error. m_5 is 0101. In POS derivation, $\bar{F}$ must be 1 at m_5. $\bar{F}(m_5) = CD + B = (0 \cdot 1) + 1 = 1$. OK. $F(m_5) = 0$. But spec says m_5 is 1. Contradiction. Spec: $H = \sum m(0, 1, 2, 5, 8, 9, 10)$. m_5 IS a 1. My POS gives 0 at m_5. So POS is wrong. Why? Grouping 0s for $\bar{F}$. 0-cells are indices NOT in $\{0, 1, 2, 5, 8, 9, 10\}$ and NOT in X $\{4, 11, 14\}$. Universe 0..15. 1s: 0,1,2,5,8,9,10. X: 4,11,14. 0s: 3,6,7,12,13,15. (6 zeros). Map:

- Row 00: 1,1,0,1 (m_0, m_1, m_3, m_2)
- Row 01: X,1,0,0 (m_4, m_5, m_7, m_6)
- Row 11: 0,0,X,0 ($m_{12}, m_{13}, m_{15}, m_{14}$)
- Row 10: 1,1,0,X (m_8, m_9, m_{11}, m_{10})

Group 0s:

- $m_3(0011), m_7(0111), m_{11}(X), m_{15}(1111)$ → Col 11 ($CD = 11$). Rows 00,01,10(X),11. Group of 4 → CD.
- $m_6(0110), m_{14}(X)$ → Row 01,11 Col 10. Group of 2 → $B\bar{D}$ ($B = 1, D = 0$).
- $m_{12}(1100), m_{13}(1101), m_{14}(X), m_{15}(1111)$ → Row 11 all cols. Group of 4 → AB.
- $m_4(X), m_6(0110)$ → Row 01 Col 00,10. Group of 2 → $\bar{A}B$ ($A = 0, B = 1$).

We cannot include m_5 in a 0-group. So group $\bar{A}B$ (row 01) is INVALID because it covers $m_5 = 1$. We can only group $m_4(X)$ and $m_6(0)$ → pair $\{4, 6\}$ → $\bar{A}B\bar{C}$? Col 00 ($C = 0$), Col 10 ($C = 1$) → C changes. Row 01 ($A = 0, B = 1$). Term $\bar{A}B$. But this group spans col 00 and 10. It includes $m_4(X)$ and $m_6(0)$. It does NOT include m_5 (col 01) or m_7 (col 11). Rectangular group in K-map must be contiguous cols. Cols 00 and 10 are adjacent (wrap). So $\{m_4, m_6\}$ is a valid pair (cols 00,10 row 01). It does not include m_5 (col 01). So group $\{m_4(X), m_6(0)\}$ → $\bar{A}B$ ($A = 0, B = 1$ constant. C changes, D=0 constant? Col 00: $C = 0, D = 0$. Col 10: $C = 1, D = 0$. $D = 0$ constant. C changes. So term $\bar{A}B\bar{D}$). Similarly $\{m_{12}(0), m_{13}(0), m_{14}(X), m_{15}(0)\}$ Row 11 all cols → AB. $\{m_3(0), m_7(0), m_{11}(X), m_{15}(0)\}$ Col 11 → CD. $\{m_6(0), m_{14}(X)\}$ Col 10 rows 01,11

→ $B\bar{D}$. {$m_{12}(0), m_8(1)$? No. $m_0(1)$? No.} Remaining 0: m_{13} covered by AB. $m_3, 7, 15$ covered by CD. m_6 covered by $B\bar{D}$ and $\bar{A}B\bar{D}$. $m_{12}, 14, 15$ covered by AB. What about m_2? 1. m_{10}? 1. $\bar{F} = CD + B\bar{D} + AB + \bar{A}B\bar{D}$. Simplify: $B\bar{D} + \bar{A}B\bar{D} = B\bar{D}$. $AB + B\bar{D} = B(A + \bar{D})$. $\bar{F} = CD + B(A + \bar{D})$. $F = (\bar{C} + \bar{D}) \cdot (\bar{B} + \bar{A}D)$. Check m_5 (0101): $\bar{C} + \bar{D} = 1 + 0 = 1$. $\bar{B} + \bar{A}D = 0 + 1 \cdot 1 = 1$. $F = 1$. Good. Check m_0 (0000): $\bar{C} + \bar{D} = 1$. $\bar{B} + \bar{A}D = 1$. $F = 1$. Good. Check m_3 (0011): $\bar{C} + \bar{D} = 0$. $F = 0$. Good. So POS is $(\bar{C} + \bar{D})(\bar{B} + \bar{A}D)$. SOP was $B\bar{D} + B\bar{C} + \bar{A}B\bar{C}$. Are they equal? SOP: $\bar{B}(\bar{C} + \bar{D}) + \bar{A}B\bar{C}$. POS: $(\bar{C} + \bar{D})(\bar{B} + \bar{A}D) = B\bar{C} + B\bar{D} + \bar{A}D\bar{C} + \bar{A}D\bar{D} = B\bar{C} + B\bar{D} + \bar{A}\bar{C}D$. SOP has $\bar{A}B\bar{C}$. POS has $\bar{A}\bar{C}D$. Check m_4 (X): SOP: $\bar{A}B\bar{C} = 1 \cdot 1 \cdot 1 = 1$. POS: $\bar{A}\bar{C}D = 1 \cdot 1 \cdot 0 = 0$. X so ok. Check m_{11} (X): 1011. SOP: $B\bar{D} = 0, B\bar{C} = 0, \bar{A}B\bar{C} = 0$. POS: $\bar{A}\bar{C}D = 0$. Ok. Check m_{14} (X): 1110. SOP: 0. POS: 0. Ok. Check m_{13} (0): 1101. SOP: 0. POS: $\bar{A}\bar{C}D = 0$. Ok. They are equivalent on care-set. So SOP: $B\bar{C} + B\bar{D} + \bar{A}B\bar{C}$. POS: $(\bar{C} + \bar{D})(\bar{B} + \bar{A}D)$. I will provide the corrected solution in the corrige box.

KEY POINTS **Key points**

- A K-map arranges minterms in Gray code so that adjacent cells (including wrap-around edges) differ in exactly one variable.
- Groups must contain 2^k cells and eliminate k variables; bigger groups give simpler terms.
- Cover all 1s with the fewest, largest groups; essential prime implicants are compulsory.
- Don't-cares (X) are optional: use them only to enlarge groups.
- Grouping the 0s and complementing yields a minimal POS.
- Always verify the result against the original truth table.

4 ANALYSING AND DESIGNING COMPLETE LOGIC SYSTEMS

In the previous sections we built the tools: Boolean algebra gave us the laws to manipulate expressions, standard forms (SOP and POS) gave us a systematic way to write any function, and the Karnaugh map gave us a fast, visual minimisation technique. We now put everything together. In the GCE Advanced Level examination you will rarely be asked to minimise an expression in isolation; you will be given a real situation — a committee vote, an alarm, a water pump — and asked to **analyse** or **design** a complete digital logic system. This section teaches you the full methodology, from a problem stated in words down to a circuit drawn with gates.

4.1 Combinational Systems: The Objects We Design

DEFINITION **Combinational logic system**

A **combinational logic system** (or combinational circuit) is a digital circuit whose outputs depend only on the present values of its inputs, and not on any past history of the inputs. The circuit has no memory: if you apply the same input combination twice, at any two different times, you always obtain the same output.

This is precisely why truth tables and Boolean expressions are sufficient to describe such circuits completely: the truth table is the full behaviour of the system. Circuits with memory (flip-flops, counters, registers) are called sequential systems and belong to a later chapter.

4.2 The Complete Design Methodology

Designing a combinational system is a disciplined process. Skipping steps is the number one cause of wrong answers in the examination.

METHOD **The six-step design procedure for combinational systems**

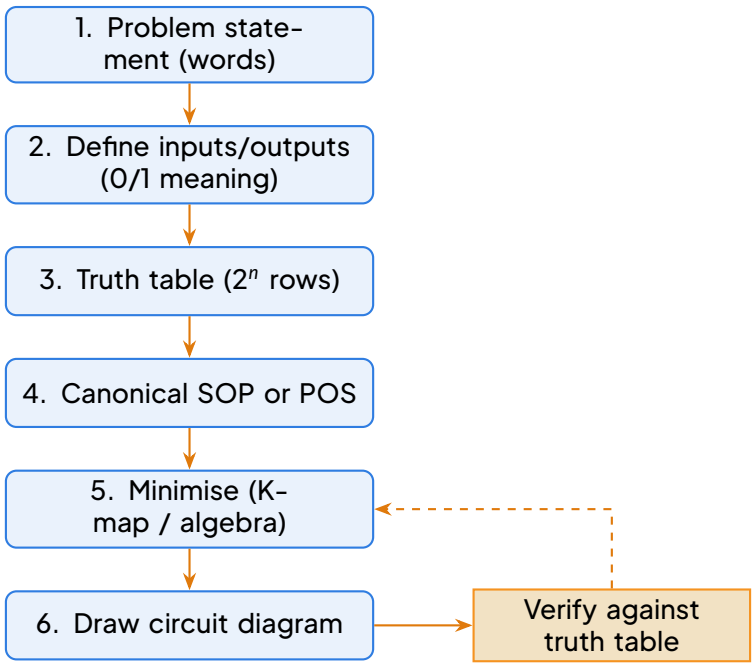
1. Understand the problem statement. Read the specification carefully and identify what the system must do.
2. Define the input and output variables. Give each variable a name and state unambiguously what logic 0 and logic 1 mean for it (e.g. $D = 1$ means “door open”, $D = 0$ means “door closed”).
3. Build the truth table. List all 2^n input combinations for n inputs, and fill in the required output for each row, directly from the specification.
4. Write the canonical form. Read off the canonical sum of products (from the rows where the output is 1) or the canonical product of sums (from the rows where the output is 0).
5. Minimise. Use a Karnaugh map (or Boolean algebra) to obtain the simplest equivalent expression.
6. Draw the circuit diagram. Translate the minimised expression into logic gates (AND, OR, NOT), possibly converting to NAND-only or NOR-only form if required.

A final, unofficial but essential step: verify the minimised circuit against the original truth table.

WARNING **Ambiguous variable definitions**

The most frequent design error is defining an input without saying what 0 and 1 mean. If

the problem says "a sensor detects the door", you must write something like: "Let $D = 1$ if the door is open, $D = 0$ if it is closed." Choosing the opposite convention gives a different (complementary) circuit. Examiners award marks for clear definitions — always state them.



Flowchart of the complete design methodology, with the verification loop.

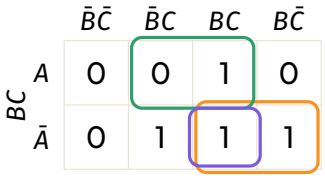
4.3 Case Study 1: A Majority Vote System

Situation. A three-member committee in a Yaoundé cooperative votes on a proposal. Each member presses a button: 1 for "yes", 0 for "no". A lamp must light when a majority (at least two) of the members vote yes.

Step 2 — Variables. Let A, B, C be the three votes (1 = yes, 0 = no), and M the output (1 = lamp on, 0 = lamp off).

Step 3 — Truth table. $M = 1$ exactly when two or more of A, B, C equal 1.

A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



Truth table of the 3-input majority function and its K-map with the three overlapping groups.

Step 4 — Canonical SOP. From the four rows where $M = 1$:

$$M = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC.$$

Step 5 — Minimisation. On the K-map, the four 1s form three overlapping pairs (the cell ABC is shared by all three groups), giving:

$$M = AB + AC + BC.$$

EXEMPLE **Algebraic check of the majority function**

We can confirm the K-map result using the consensus theorem:

$$\begin{aligned} M &= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC \\ &= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC + ABC + ABC \\ &= BC(\bar{A} + A) + AC(\bar{B} + B) + AB(\bar{C} + C) \\ &= BC + AC + AB. \end{aligned}$$

Notice the trick: duplicating ABC (legal since $X + X = X$) lets it pair with each of the other three terms — exactly what the overlapping K-map groups show.

Step 6 — Circuit. Three 2-input AND gates feeding one 3-input OR gate: $M = AB + AC + BC$.

DID YOU KNOW? **Majority voting in real systems**

Majority voting is not just a classroom example. Safety-critical computers (in aircraft and space systems) often run three identical computers in parallel and take a majority vote of their outputs, so that if one computer fails, the two correct ones outvote it. This is called triple modular redundancy.

4.4 Case Study 2: A Car Alarm System

Situation. A car alarm must sound ($S = 1$) when the **door is open while the engine is off**, or when the **key is left in the ignition while the engine is off**.

Variables. $D = 1$: door open; $E = 1$: engine on; $K = 1$: key in ignition. Output $S = 1$: alarm sounds.

Truth table and canonical form. $S = 1$ when $E = 0$ and ($D = 1$ or $K = 1$). The rows giving $S = 1$ are $(D, E, K) = (1, 0, 0), (0, 0, 1), (1, 0, 1)$:

$$S = D\bar{E}\bar{K} + \bar{D}\bar{E}K + D\bar{E}K.$$

K-map minimisation. Placing these three 1s on a K-map with D down the side and EK across the top, the column $\bar{E}\bar{K}$… more precisely, the three cells group into one pair $D\bar{E}$ and one pair $\bar{E}K$:

$$S = \bar{E}D + \bar{E}K = \bar{E}(D + K).$$

The factored form $\bar{E}(D+K)$ needs only one NOT, one OR and one AND gate — three gates instead of the five or six a naive reading would suggest.

4.5 Case Study 3: The Water-Tank Pump Controller (Complete Design)

This classic problem appears frequently in GCE papers. We solve it fully, from specification to a NAND-only circuit.

WORKED EXAMPLE 6 **Water-tank pump controller**

Statement. A pump P fills a water tank. Two level sensors are fitted: L at the low level and H at the high level. A sensor outputs 1 when it is covered by water, 0 when dry. The pump must run ($P = 1$) when the water level is below the low sensor (tank nearly empty), and must stop ($P = 0$) when the water reaches or passes the high sensor (tank full). When the level is between the two sensors, the pump keeps its previous behaviour — but since this is a combinational design exercise, the specification tells us to treat the "between" state as: pump runs only if it was filling, i.e. we use a manual override switch M ($M = 1$: force pump on) so that $P = 1$ when the tank is below low level or when the level is between the sensors and the override is on, provided the high sensor is dry.

SOLUTION

Step 2 — Variables. $L = 1$: low sensor covered; $H = 1$: high sensor covered; $M = 1$: manual override on; $P = 1$: pump running.
Step 3 — Truth table. Note that $L = 0, H = 1$ is physically impossible (water cannot cover the high sensor while the low one is dry): mark it as a don't care (×).

L	H	M		P
0	0	0	below low	1
0	0	1	below low	1
0	1	0	impossible	×
0	1	1	impossible	×
1	0	0	between	0
1	0	1	between, override	1
1	1	0	full	0
1	1	1	full	0

Step 4 — Canonical SOP (rows where $P = 1$):

$$P = \bar{L}\bar{H}\bar{M} + \bar{L}\bar{H}M + L\bar{H}M.$$

Step 5 — K-map minimisation. With L down the side and HM across the top (order 00, 01, 11, 10), the 1s are at $\bar{L}\bar{H}\bar{M}$, $\bar{L}\bar{H}M$, $L\bar{H}M$, with don't cares in the row $\bar{L}H\cdot$. Grouping:

- The pair $\{\bar{L}\bar{H}\bar{M}, \bar{L}\bar{H}M\}$ gives $\bar{L}\bar{H}$.
- The pair $\{\bar{L}\bar{H}M, L\bar{H}M\}$ gives $\bar{H}M$.

$$P = \bar{L}\bar{H} + \bar{H}M = \bar{H}(\bar{L} + M).$$

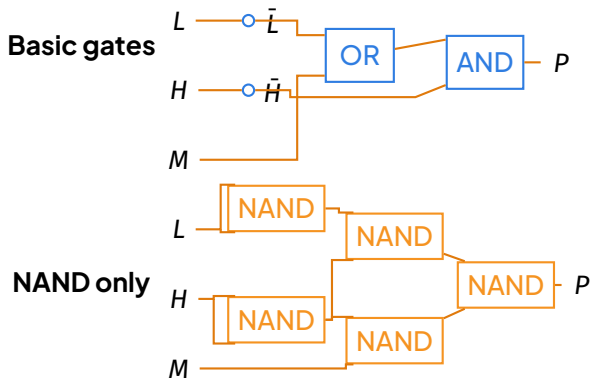
Interpretation. The pump runs when the tank is not full ($\bar{H}$) and either the water is below the low sensor ($\bar{L}$) or the override is on (M). This matches the specification exactly.
Step 6a — Basic-gate circuit. One NOT for H, one NOT for L, one OR ($\bar{L} + M$), one AND.
Step 6b — NAND-only circuit. Using double complementation and De Morgan:

$$P = \bar{H}(\bar{L} + M) = \overline{\overline{\bar{H}(\bar{L} + M)}} = \overline{\overline{\bar{H}} + \overline{\bar{L} + M}}.$$

A cleaner NAND-only route uses the SOP form $P = \bar{L}\bar{H} + \bar{H}M$:

$$P = \overline{\overline{\bar{L}\bar{H}} \cdot \overline{\bar{H}M}},$$

which is a two-level NAND structure: two NAND gates producing $\overline{\bar{L}\bar{H}}$ and $\overline{\bar{H}M}$, feeding a final NAND gate (with $\bar{L}$ and $\bar{H}$ obtained from NAND gates with tied inputs acting as NOT gates).



Water-tank pump controller: basic-gate version $P = \bar{H}(\bar{L} + M)$ (top) and NAND-only version $P = \overline{\bar{L}\bar{H}} \cdot \overline{\bar{H}M}$ (bottom). Tied NAND inputs act as NOT gates.

WARNING **Verify the minimised circuit**

A minimised expression is worthless if it no longer matches the specification. After minimising, pick at least two or three input combinations — especially the ones that produced the output 1 — and check them against the original truth table. In the pump example, test $LHM = 000$ (must give $P = 1$) and $LHM = 110$ (must give $P = 0$). Don't-care rows are the only rows allowed to differ.

4.6 Analysing an Existing Circuit

Analysis is the reverse of design: you are given the circuit and must find out what it does.

METHOD **Analysing a combinational circuit**

1. Label the output of every gate with its expression, working from inputs to outputs.
2. Write the final output as a Boolean expression in the input variables.
3. Build the truth table of this expression (evaluate it for all 2^n input combinations).
4. Minimise the expression (K-map or algebra).
5. If the minimised form is simpler than the original circuit, propose a cheaper equivalent circuit and prove equivalence by showing both give the same minimised expression (or the same truth table).

WORKED EXAMPLE 7 **Analysing and improving a circuit**

Statement. A circuit implements $F = AB + A\bar{B} + \bar{A}B$ using three AND gates, two NOT gates and one 3-input OR gate (6 gates). Analyse it and propose a cheaper equivalent.

SOLUTION

Step 1–2. The expression is already given: $F = AB + A\bar{B} + \bar{A}B$.
Step 3 — Truth table. Evaluating for the four combinations of (A, B): $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 1$.
Step 4 — Minimise.

$$F = AB + A\bar{B} + \bar{A}B = A(B + \bar{B}) + \bar{A}B = A + \bar{A}B = A + B,$$

using the absorption law $X + \bar{X}Y = X + Y$.
Step 5 — Cheaper circuit. $F = A + B$ is a single OR gate. The original 6-gate circuit was simply an over-complicated OR gate! Both have the same truth table, so they are functionally equivalent.

METHOD **Proving two circuits equivalent**

Two circuits are **functionally equivalent** if they produce the same output for every input combination. To prove equivalence:

1. Method A (truth tables): build the truth table of each circuit; if the output columns are identical, the circuits are equivalent. This always works but is long for many inputs.
2. Method B (minimisation): derive each circuit's expression and minimise both (K-map or Boolean algebra). If both reduce to the same minimal expression, the circuits are equivalent.

In the exam, state clearly which method you use and show the two identical final results.

4.7 Technology Note: Does Minimisation Still Matter?

DID YOU KNOW? **From SSI chips to programmable logic**

In the 1970s, engineers built circuits from **SSI** (Small-Scale Integration) chips such as the 7400 series, each containing a handful of gates. Every gate saved meant a cheaper, cooler, more reliable board — minimisation was essential. Today, designers often use **programmable logic devices** (PLDs, FPGAs) where the circuit is described in a hardware description language and a software tool performs the minimisation automatically. So why learn K-maps? Because the reasoning is unchanged: you must still define variables precisely, build correct truth tables, and understand what the tool is doing — and GCE exam-

iners test exactly that understanding. Minimisation also builds the logical thinking needed for algorithm design later in your studies.

4.8 Common Examination Pitfalls

WARNING **Frequent errors in analysis and design questions**

- Mislabelling K-map axes. The columns/rows must follow Gray code order 00, 01, 11, 10 — not binary counting order 00, 01, 10, 11. A wrong order makes valid groups look scattered.
- Dropping literals. When a variable appears both complemented and uncomplemented inside a group, it is eliminated; students often keep it by mistake. Each group of 2^k cells eliminates exactly k variables.
- Forgetting to verify. Always substitute a few input combinations back into the original truth table or specification. Two minutes of checking saves the whole question.
- Ignoring don't cares. Physically impossible combinations (like $\bar{L}H$ in the pump problem) should be marked × and used only if they enlarge a group.
- Ambiguous 0/1 definitions. Repeat of the golden rule: state what 0 and 1 mean for every variable before drawing the truth table.

EXERCISE 4 **Design a security light**

A security light L for a house in Bamenda must turn on when it is dark ($D = 1$) and motion is detected ($M = 1$), or when the manual switch S is on ($S = 1$) regardless of the other conditions. Define your variables, build the truth table, write the canonical SOP, minimise with a K-map, and draw the final circuit.

SOLUTION **Exercise 1**

Variables: $D = 1$ dark, $M = 1$ motion detected, $S = 1$ manual switch on; $L = 1$ light on.
From the specification, $L = 1$ whenever $S = 1$ (4 rows) or when $D = 1$ and $M = 1$ with $S = 0$ (1 row). Canonical SOP:

$$L = \bar{S}DM + S\bar{D}\bar{M} + S\bar{D}M + SD\bar{M} + SDM.$$

K-map (rows S, columns DM in order 00, 01, 11, 10): the entire row $S = 1$ is a group of four giving S; the cell $\bar{S}DM$ groups with SDM giving DM. Hence

$$L = S + DM.$$

Circuit: one AND gate (DM) feeding one OR gate with S. Verification: $DMS = 110$ gives $L = 0 + 1 \cdot 1 = 1$ ✓; $DMS = 000$ gives $L = 0$ ✓.

KEY POINTS **Key points**

- A **combinational system** has outputs depending only on present inputs; its truth table fully describes it.
- Follow the six-step design procedure: problem → variables (with 0/1 meanings) → truth table → canonical form → minimisation → circuit — then verify.
- To analyse a circuit: derive its expression, tabulate it, minimise, and propose a cheaper equivalent.
- Two circuits are equivalent if they share the same truth table or minimise to the same expression.
- Classic patterns to recognise: majority vote ($AB+AC+BC$), alarm conditions (factorised forms like $\bar{E}(D + K)$), and controllers with don't-care states (pump problem) leading to NAND-only implementations.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This integration activity is a complete worked activity where you must:

- define variables and build the full truth table;
- write the canonical SOP and simplify algebraically using Boolean laws;
- confirm the simplification with a 4–variable K-map;
- draw the circuit with basic gates, then convert it to NAND-only;
- compare gate counts and justify the cheapest reliable solution.

5.2 Situation

You are the project team of the OnBuch+ app. Your mission is to design an automatic water-pump controller for a school in Yaoundé. The school's water pump must start (P=1) when the tank level is low (L=1) and ENEO power is available (E=1), or when the watchman presses a manual override (M=1) and power is available; however a safety float (S=1, tank full) must always stop the pump. In teams, students: (1) define variables and build the full truth table; (2) write the canonical SOP; (3) simplify algebraically using the laws of Section 1; (4) confirm with a 4–variable K-map; (5) draw the circuit with basic gates, then convert it to NAND-only; (6) compare gate counts and justify the cheapest reliable solution. Groups present and cross-verify each other's truth tables.

5.3 Tasks

- **Task 1:** Define the four input variables L, E, M, S and the output P, stating clearly what the values 0 and 1 mean for each.
- **Task 2:** Translate the statement into a first Boolean expression for P, then build the complete 16–row truth table.
- **Task 3:** Extract the canonical sum-of-products (SOP) form of P from the truth table and list the minterms.
- **Task 4:** Simplify the canonical SOP algebraically, justifying each step by the name of the Boolean law used.
- **Task 5:** Plot P on a 4–variable Karnaugh map, group the 1s, and confirm the minimal expression.
- **Task 6:** Draw the logic circuit of the minimal expression using basic gates (NOT, AND, OR).
- **Task 7:** Convert the circuit to a NAND-only implementation using De Morgan's theorem.
- **Task 8:** Count the gates in both implementations, compare the costs, and justify the cheapest reliable solution. Then cross-verify your truth table with another group.

5.4 Model answer

Task 1: definition of the variables.

We choose four Boolean inputs and one output, all active-high:

Variable	Meaning
L	$L = 1$: tank level is low; $L = 0$: level is not low
E	$E = 1$: ENEO power available; $E = 0$: power cut
M	$M = 1$: watchman presses the manual override; $M = 0$: not pressed
S	$S = 1$: safety float up (tank full); $S = 0$: tank not full
P	$P = 1$: pump runs; $P = 0$: pump stopped

Task 2: first expression and truth table.

The statement gives: the pump runs when (L AND E) OR (M AND E), but the safety float must always stop it, i.e. the whole condition is valid only when $S = 0$:

$$P = \overline{S} \cdot \left(L \cdot E + M \cdot E\right).$$

The full truth table is:

L	E	M	S	P
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Note that whenever $S = 1$ (tank full), $P = 0$ for every combination of the other inputs: the safety condition dominates, exactly as required.

Task 3: canonical SOP.

Reading the rows where $P = 1$ (rows 6, 13 and 15), the canonical sum of products is:

$$P = \overline{L}EM\overline{S} + LE\overline{M}\,\overline{S} + LEM\overline{S} = \sum m(6, 12, 14)$$

with the variable order (L, E, M, S).

Task 4: algebraic simplification.

$P = \overline{L}EM\overline{S} + LE\overline{M}\,\overline{S} + LEM\overline{S}$	canonical SOP
$= \overline{L}EM\overline{S} + LEM\overline{S} + LEM\overline{S} + LE\overline{M}\,\overline{S}$	idempotence $X + X = X$
$= EM\overline{S}(\overline{L} + L) + LE\overline{S}(M + \overline{M})$	factorisation (distributivity)
$= EM\overline{S} \cdot 1 + LE\overline{S} \cdot 1$	complement law $X + \overline{X} = 1$
$= EM\overline{S} + LE\overline{S}$	identity $X \cdot 1 = X$
$= E\overline{S}(M + L)$	factorisation (distributivity)

Hence the minimal form:

$$\boxed{P = E \cdot \overline{S} \cdot (L + M)}$$

This matches the common-sense reading: power must be available, the tank must not be full, and either the level is low or the watchman overrides.

Task 5: Karnaugh map confirmation.

Using rows LE and columns MS in Gray-code order:

$LE \backslash MS$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	0	0	1
10	1	0	0	1

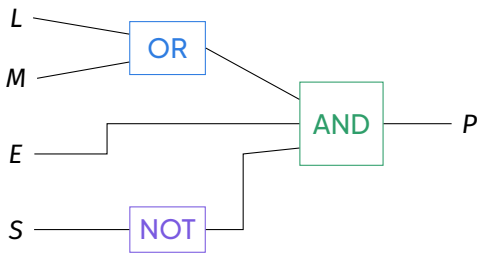
Groupings:

- the four 1s in columns $MS = 00$ and $MS = 10$ (i.e. $S = 0$) with $E = 1$ and $L = 1$ give the term $LE\overline{S}$;
- the 1s at $(LE = 11, MS = 10)$ and $(LE = 01, MS = 10)$ share $E = 1, M = 1, S = 0$: pairing the cell $m(6)$ with $m(14)$ gives $EM\overline{S}$.

So the K-map gives $P = LE\overline{S} + EM\overline{S} = E\overline{S}(L + M)$, confirming the algebraic result.

Task 6: circuit with basic gates.

The minimal expression needs: one NOT gate (for $\overline{S}$), one OR gate $(L + M)$, and one 3-input AND gate (or two 2-input AND gates) combining E, $\overline{S}$ and $(L + M)$.



Minimal circuit: $P = E \cdot \overline{S} \cdot (L + M)$ with basic gates.

Task 7: NAND-only conversion.

Apply De Morgan's theorem. Start from $P = E\overline{S}(L + M)$. Replace the OR by its NAND-NOT form:

$$L + M = \overline{\overline{L} \cdot \overline{M}} = \overline{\overline{L \cdot L} \cdot \overline{M \cdot M}}$$

so a 2-input NAND with both inputs tied together acts as a NOT. Then:

$$P = \overline{\overline{E\overline{S}(L + M)}} = \overline{\overline{E} + S + \overline{(L + M)}}$$

but the cleanest construction is the direct two-level NAND transformation: complement each input of the final AND and use a NAND:

$$P = \overline{\overline{E \cdot \overline{S} \cdot (L + M)}} = \text{NAND}\left(\overline{E}, S, \overline{L + M}\right).$$

Concretely the NAND-only circuit uses:

- one NAND as inverter for S (inputs tied): gives $\overline{S}$;
- one NAND for $\overline{L \cdot M}$, then one NAND as inverter: gives $L + M$;
- one 3-input NAND with inputs $\overline{E}$ (from a NAND-inverter on E), S, and $\overline{L + M}$: its output is P.

In total: 6 two-input NAND gates (counting the 3-input NAND as two 2-input equivalents).

Task 8: comparison and justification.

Criterion	Basic gates	NAND-only
Gate count	1 NOT + 1 OR + 1 AND (3 gates, 3 different IC types)	6 NAND gates (one single IC type)
Canonical SOP circuit	3 NOT + 3 AND(4-in) + 1 OR(3-in): far more gates	even larger after conversion
Reliability / maintenance	needs several IC families in stock	one IC family (e.g. quad NAND), easy to replace in Yaoundé
Cost	low	lowest in practice: one chip can host all gates

The minimised basic-gate circuit uses the fewest gates, but the NAND-only version uses a single type of integrated circuit: a single quad-NAND chip is enough, which is cheaper to buy, easier to maintain and more reliable for a school. This is the solution we recommend.

WARNING **Cross-verification**

When groups compare truth tables, the most frequent error is a row where $S = 1$ but P was left at 1. Remember: the safety float must always stop the pump, so the whole column $S = 1$ must give $P = 0$. Another classic mistake is forgetting that the K-map columns follow Gray code (00, 01, 11, 10), not binary order.

KEY POINTS **Design method to remember**

1. Define variables and their active levels.
2. Build the truth table from the statement.
3. Write the canonical SOP.
4. Simplify by Boolean algebra and by K-map; both must agree.
5. Draw the circuit, convert to NAND-only if required, and justify the final technological choice.

6 METHODS AND WORKED EXAMPLES

7 ANALYSING DIGITAL LOGIC SYSTEMS

7.1 Boolean Algebra: Variables, Operations and Fundamental Laws

DEFINITION **Boolean variable, literal and expression**

A Boolean variable takes only two values: **0** (FALSE, LOW, 0 V) or **1** (TRUE, HIGH, V_{CC}). A literal is a variable or its complement (e.g. A, $\bar{A}$). A Boolean expression combines literals with the three basic operations:

- AND (conjunction): written $A \cdot B$ or AB; output 1 iff all inputs are 1.
- OR (disjunction): written $A + B$; output 1 iff at least one input is 1.
- NOT (complementation): written $\bar{A}$ or A'; output is the opposite of the input.

PROPERTY **Fundamental laws of Boolean algebra**

The following identities hold for all $A, B \in \{0, 1\}$. All are examinable and must be quoted by name in algebraic proofs.

Law	OR form	AND form
Identity	$A + 0 = A$	$A \cdot 1 = A$
Null (Domination)	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotence	$A + A = A$	$A \cdot A = A$
Complement	$A + \bar{A} = 1$	$A \cdot \bar{A} = 0$
Involution	$\overline{\bar{A}} = A$	—
Commutativity	$A + B = B + A$	$A \cdot B = B \cdot A$
Associativity	$(A + B) + C = A + (B + C)$	$(AB)C = A(BC)$
Distributivity	$A + (BC) = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption	$A + AB = A$	$A(A + B) = A$
Redundancy (Consensus)	$A + \bar{A}B = A + B$	$A(\bar{A} + B) = AB$
De Morgan	$\overline{A + B} = \bar{A}\bar{B}$	$\overline{AB} = \bar{A} + \bar{B}$

KEY POINTS **Duality principle**

Every Boolean identity remains valid if you swap + with ·, 0 with 1, and leave variables unchanged. The two columns in the table above are duals of each other.

METHOD **Proving a Boolean identity algebraically**

1. List the laws you intend to use (the toolbox above).
2. Start from the more complex side (usually the side with more literals or bars).
3. Transform step by step: write each new line directly under the previous one.
4. Justify every line by naming the law used — GCE marks are awarded for named laws.
5. Useful reflexes: duplicate a term ($X = X+X$); expand with $X = X(Y+\bar{Y})$; factorise common factors.
6. Optional check: evaluate both sides on one or two rows of a truth table.

WORKED EXAMPLE 8 **Proving the redundancy theorem**

Prove, using only the basic laws, that $A + \bar{A}B = A + B$. State the law at each step.

SOLUTION

Start from the left-hand side (more complex):

$A + \bar{A}B = A \cdot 1 + \bar{A}B$	(Identity: $A = A \cdot 1$)
$= A(B + \bar{B}) + \bar{A}B$	(Complement: $1 = B + \bar{B}$)
$= AB + A\bar{B} + \bar{A}B$	(Distributivity)
$= AB + AB + A\bar{B} + \bar{A}B$	(Idempotence: $AB = AB + AB$)
$= (AB + A\bar{B}) + (AB + \bar{A}B)$	(Associativity)
$= A(B + \bar{B}) + B(A + \bar{A})$	(Distributivity, factoring)
$= A \cdot 1 + B \cdot 1$	(Complement)
$= A + B$	(Identity)

Hence $A + \bar{A}B = A + B$. Check: if $A = 1$, both sides = 1; if $A = 0$, both sides = B. Identity holds.

WARNING **Common traps in algebraic proofs**

- Do not write $A + B = A$ (false unless $B = 0$). Absorption is $A + AB = A$, not $A + B = A$.
- De Morgan: $\overline{A + B} = \bar{A}\bar{B}$ (bar breaks, sign changes). A frequent error is $\overline{A + B} = \bar{A} + \bar{B}$.
- Distributivity has two forms: $A(B + C) = AB + AC$ (like algebra) AND $A + BC = (A + B)(A + C)$ (unlike algebra). Learn both.
- Justification: "Simplifying" is not a law name. Write "Absorption", "Distributivity", etc.

METHOD **Algebraic simplification of a sum-of-products**

1. Apply De Morgan first to push bars down to single variables.
2. Multiply out only when necessary; prefer factorising common literals.
3. Hunt for patterns:
   - $X + \bar{X} = 1$ (complement)
   - $X + XY = X$ (absorption)
   - $X + \bar{X}Y = X + Y$ (redundancy)
   - $XY + X\bar{Y} = X$ (combining)
4. Consensus theorem: $XY + \bar{X}Z + YZ = XY + \bar{X}Z$ (the term YZ is redundant).
5. Count literals before and after: a good simplification strictly reduces the literal count.

WORKED EXAMPLE 9 **Simplifying an alarm expression**

A school laboratory alarm in Bamenda is described by

$$F = A\bar{B}C + A\bar{B}\bar{C} + ABC + \bar{A}BC.$$

Simplify F fully, naming each law, and state the gate count of the simplified circuit.

SOLUTION

Group terms sharing common factors:

$$
\begin{aligned}
F &= A\bar{B}C + A\bar{B}\bar{C} + ABC + \bar{A}BC \\
&= A\bar{B}(C + \bar{C}) + BC(A + \bar{A}) && \text{(Distributivity)} \\
&= A\bar{B} \cdot 1 + BC \cdot 1 && \text{(Complement)} \\
&= A\bar{B} + BC && \text{(Identity)}
\end{aligned}
$$

Simplified expression: $F = A\bar{B} + BC$. Gate count: 1 NOT ($\bar{B}$), 2 two-input ANDs, 1 two-input OR = 4 gates. Original needed 4 three-input ANDs, 1 three-input OR, and inverters → much larger. Interpretation: Alarm sounds if (A active AND B inactive) OR (B AND C both active).

DID YOU KNOW? **Claude Shannon and the birth of digital design**

In 1937, Claude Shannon's master's thesis A Symbolic Analysis of Relay and Switching Circuits proved that Boolean algebra could model electrical switching circuits. This founded modern digital design. In Cameroon, the first digital telephone exchanges (e.g. Camtel's E10B switches in the 1990s) relied directly on this theory.

7.2 Standard Forms and Logic Minimisation

DEFINITION **Minterm, Maxterm, Canonical SOP and POS**

For n variables, a minterm m_i is a product of all n literals (each variable appears once, complemented if 0, uncomplemented if 1 in row i). A maxterm M_i is a sum of all n literals (complemented if 1, uncomplemented if 0 in row i).

- Canonical SOP (Sum of Products): $F = \Sigma m$(indices where $F = 1$) = OR of minterms.
- Canonical POS (Product of Sums): $F = \Pi M$(indices where $F = 0$) = AND of maxterms.

METHOD **From specification to truth table and canonical SOP**

1. Identify inputs/outputs; assign letters.
2. List all 2^n combinations in binary counting order (000, 001, ...).
3. Fill output column from the problem statement.
4. Select rows where output = 1.

5. For each selected row, write the minterm (variable if 1, variable if 0).
6. OR the minterms → canonical SOP.
7. For canonical POS: select rows where output = 0, write maxterms (variable if 0, variable if 1), AND them.

WORKED EXAMPLE 10 **Majority vote of three teachers**

Three teachers A, B, C vote; decision $F = 1$ if at least two vote yes. Give truth table and canonical SOP.

SOLUTION

$2^3 = 8$ combinations. $F = 1$ for rows with two or three 1s:

A	B	C	F	minterm
0	0	0	0	—
0	0	1	0	—
0	1	0	0	—
0	1	1	1	$\bar{A}BC$
1	0	0	0	—
1	0	1	1	$A\bar{B}C$
1	1	0	1	$AB\bar{C}$
1	1	1	1	ABC

Canonical SOP: $F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC = \Sigma m(3,5,6,7)$. (Simplifies to $AB + AC + BC$, shown later.)

METHOD **Converting between SOP, POS and Σ/Π notation**

1. Number truth table rows by binary value ($ABC = 101_2 = 5$).
2. SOP: $F = \Sigma m$(list of indices with $F = 1$). Write each minterm.
3. POS: $F = \Pi M$(list of indices with $F = 0$). Write each maxterm (complement if bit=1).
4. Check: Σ indices $\cup$ Π indices $= \{0, 1, \ldots, 2^n - 1\}$ (disjoint, complete).
5. To expand a non-canonical term (e.g. AB) into minterms: multiply by $(C + \bar{C})$ for each missing variable.

WORKED EXAMPLE 11 **SOP and POS from Σ notation**

Given $F(A, B, C) = \Sigma m(1.3, 5.6)$. Write canonical SOP and canonical POS.

SOLUTION

Canonical SOP. Convert indices to 3-bit binary; 1 → variable, 0 → complement:

$$m_1 = 001 \rightarrow \bar{A}\bar{B}C, \quad m_3 = 011 \rightarrow \bar{A}BC,$$
$$m_5 = 101 \rightarrow A\bar{B}C, \quad m_6 = 110 \rightarrow AB\bar{C}.$$

$F = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + AB\bar{C}$.

Canonical POS. Zero rows: $\{0, 2, 4, 7\}$ (complement of $\{1, 3, 5, 6\}$). Maxterm: complement if bit=1.

$$M_0 = 000 \rightarrow (A + B + C), \quad M_2 = 010 \rightarrow (A + \bar{B} + C),$$
$$M_4 = 100 \rightarrow (\bar{A} + B + C), \quad M_7 = 111 \rightarrow (\bar{A} + \bar{B} + \bar{C}).$$

$F = (A + B + C)(A + \bar{B} + C)(\bar{A} + B + C)(\bar{A} + \bar{B} + \bar{C})$.

WARNING **Minterm vs Maxterm convention**

- Minterm (m_i): variable appears uncomplemented if its value is 1, complemented if 0.
- Maxterm (M_i): variable appears complemented if its value is 1, uncomplemented if 0.
- Mnemonic: “Maxterm is the opposite of minterm”.

METHOD **Karnaugh map minimisation (up to 4 variables)**

1. Choose map size: 2 var → 2 × 2; 3 var → 2 × 4; 4 var → 4 × 4.
2. Label axes in Gray code: 00, 01, 11, 10 (only one bit changes between neighbours).
3. Place 1s from truth table / minterm list; 0s or blanks elsewhere.
4. Group 1s in rectangles of size 1, 2, 4, 8, … (powers of two).
   - Map wraps: left↔right, top↔bottom, four corners (4-var).
   - Make groups as large as possible; use as few groups as possible.
   - Every 1 must be covered; overlapping allowed.
5. Read each group: keep variables that do not change inside the group. Write uncomplemented if fixed at 1, complemented if fixed at 0.

6. OR the group terms → minimal SOP.

WORKED EXAMPLE 12 **K-map minimisation of majority function**

Minimise $F(A, B, C) = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$ using a K-map.

SOLUTION

Place 1s at indices 3,5,6,7 (rows 011, 101, 110, 111) on a 3-var map (A rows, BC columns Gray code).

BC	00	01	11	10
$A = 0$	0	0	1	0
$A = 1$	0	1	1	1

Group 1: BC
Group 2: AC
Group 3: AB

K-map of majority function $F(A, B, C)$ with three optimal groups of two.

Reading groups:

- Orange (column $BC = 11$, both A): A changes, $B = 1$, $C = 1$ fixed → BC.
- Blue (row $A = 1$, columns 01, 11): B changes, $A = 1$, $C = 1$ fixed → AC.
- Green (row $A = 1$, columns 11, 10): C changes, $A = 1$, $B = 1$ fixed → AB.

Minimal SOP: $F = AB + AC + BC$. Gain: 4 minterms × 3 literals → 3 terms × 2 literals. Circuit: 3 two-input ANDs + 1 three-input OR (vs 4 three-input ANDs + 1 four-input OR).

EXPERIMENT **Don't-care conditions (X) in K-maps**

Some input combinations never occur (e.g. BCD codes 1010–1111). Mark these X in the K-map. Treat X as 1 only if it helps enlarge a group; otherwise treat as 0. This yields even simpler circuits. Examinable for 4-variable maps.

EXERCISE 5 **Practice: Algebraic simplification and K-map**

1. Simplify $F = A\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}\bar{B}C + ABC$ using Boolean laws. Name each law.
2. Use a K-map to minimise $F(A, B, C, D) = \Sigma m(0, 1, 2, 5, 8, 9, 10)$.

3. A function is $F = \Pi M(0.3, 4.7)$. Write the canonical SOP and minimise it with a K-map.

SOLUTION **Exercise 1**

$$\begin{aligned} F &= A\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}\bar{B}C + ABC \\ &= \bar{C}(A\bar{B} + \bar{A}B) + C(\bar{A}\bar{B} + AB) \quad \text{(Distributivity)} \\ &= \bar{C}(A \oplus B) + C(\overline{A \oplus B}) \quad \text{(XOR pattern)} \\ &= A \oplus B \oplus C \quad \text{(Parity function)} \end{aligned}$$

Minimal SOP requires 3 two-input XOR gates (or 4 ANDs + 3 ORs + 3 NOTs if only AND/OR/NOT allowed).

SOLUTION **Exercise 2**

K-map (4 var, AB rows, CD cols Gray code). 1s at 0,1,2,5,8,9,10. Groups: (0.1,8.9) → $\bar{B}\bar{D}$; (0.2,8.10) → $\bar{A}\bar{C}$; (1,5) → $\bar{A}\bar{B}C$? Wait. Better grouping: Quad (0.1,8.9): A changes, $B = 0$, C changes, $D = 0$? No. Rows: 00,01,11,10. Cols: 00,01,11,10. Indices: 0(0000), 1(0001), 2(0010), 5(0101), 8(1000), 9(1001), 10(1010). Map:

CD \ AB	00	01	11	10
00	1	0	0	1
01	1	1	0	0
11	0	0	0	0
10	1	0	0	1

Groups:

- Corners (0.2,8.10): $\bar{B}\bar{D}$ (B=0, D=0).
- Top row pair (0,1): $\bar{A}\bar{B}\bar{C}$.
- Bottom row pair (8,9): $A\bar{B}\bar{C}$.
- Pair (1,5): $\bar{A}\bar{C}D$? 1(0001), 5(0101) → $\bar{A}\bar{C}D$.

Minimal: $\bar{B}\bar{D} + \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + \bar{A}\bar{C}D$? No, $\bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} = \bar{B}\bar{C}$. So $F = \bar{B}\bar{D} + \bar{B}\bar{C} + \bar{A}\bar{C}D$. Check literals: 2+2+3 = 7. Alternative: Pair (0,8) → $\bar{B}\bar{C}\bar{D}$, Pair (2,10) → $\bar{B}C\bar{D}$ → $\bar{B}\bar{D}$. Pair (0,1) → $\bar{A}\bar{B}\bar{C}$, Pair (8,9) → $A\bar{B}\bar{C}$ → $\bar{B}\bar{C}$. Pair (1,5) → $\bar{A}\bar{C}D$. Result: $F = \bar{B}\bar{D} + \bar{B}\bar{C} + \bar{A}\bar{C}D$.

SOLUTION **Exercise 3**

$F = \Pi M(0.3, 4.7) \implies F = \Sigma m(1.2, 5.6)$. Minterms: 1(001) = $\bar{A}\bar{B}C$, 2(010) = $\bar{A}B\bar{C}$, 5(101) = $A\bar{B}C$, 6(110) = $AB\bar{C}$. $F = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}C + AB\bar{C}$. K-map (3 var): 1s at 1,2,5,6. Groups: (1,5) vertical → $\bar{B}C$; (2,6) vertical → $B\bar{C}$. Minimal SOP: $F = \bar{B}C + B\bar{C} = B \oplus C$.

7.3 Analysing and Designing Complete Logic Systems

METHOD **Analysing a given logic circuit**

1. Label every gate output with its Boolean expression, working from inputs forward.
2. Combine to obtain the global output function F.
3. Simplify F (laws or K-map).
4. Build truth table if required, using the simplified expression (faster, fewer errors).
5. Interpret: describe in words what the circuit does; identify enable/inhibit signals.
6. Redraw the minimal equivalent circuit if asked.

WORKED EXAMPLE 13 **Analysing a two-level AND-OR circuit**

Circuit: inputs A, B, C. G_1=AND(A, B); G_2=AND($A, \bar{C}$); G_3=OR(G_1, G_2). Find F, simplify, describe function, draw cheaper circuit.

SOLUTION

Step 1 – Gate expressions: $G_1 = AB$, $G_2 = A\bar{C}$. Step 2 – Global function: $F = G_1 + G_2 = AB + A\bar{C}$. Step 3 – Simplify: $F = A(B + \bar{C})$ (Distributivity). Step 4 – Interpretation: A is an enable signal. Output is 1 iff A = 1 AND (B = 1 OR C = 0). If A = 0, output forced to 0. Step 5 – Cheaper circuit: 1 NOT (C), 1 OR ($B + \bar{C}$), 1 AND (with A) = 3 gates, 2 levels delay. Original: 1 NOT, 2 ANDs, 1 OR = 4 gates, 2 levels.

METHOD **Full design flow: specification → minimal circuit**

1. Analyse specification: list inputs, outputs, derive truth table.
2. Extract SOP: OR of minterms for rows where output = 1.

3. Minimise: K-map (preferred for 3–4 variables) or Boolean laws.
4. Draw circuit: NOT gates first (for complemented literals), then AND gates (one per product term), then one OR gate (SOP) – or dual for POS.
5. Verify: test 2–3 input combinations against original truth table.

WORKED EXAMPLE 14 **Water-pump controller for a school tank**

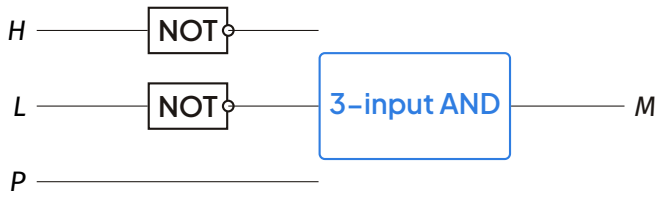
A school in Douala has a tank with sensors: H (High level, 1=full), L (Low level, 1=supply empty), P (Pressure, 1=available). Pump M runs ($M = 1$) when tank not full ($H = 0$) AND pressure available ($P = 1$), but never when supply empty ($L = 1$). Design minimal circuit.

SOLUTION

Step 1 – Truth table. $M = 1$ only for $H = 0, L = 0, P = 1$ (row 1).

H	L	P	M
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Step 2 – SOP: Single minterm $m_1 = \bar{H}\bar{L}P$. $M = \bar{H}\bar{L}P$. Step 3 – Minimisation: Only one 1 in K-map → already minimal. Step 4 – Circuit (using TikZ):



Minimal pump controller: two inverters and one 3-input AND gate.

Step 5 – Verification:

- $H = 0, L = 0, P = 1$: $\bar{H}\bar{L}P = 1 \cdot 1 \cdot 1 = 1$ (pump runs ✓).
- $L = 1$ (empty supply): $\bar{L} = 0 \Rightarrow M = 0$ (safety ✓).

- $H = 1$ (tank full): $\bar{H} = 0 \Rightarrow M = 0$ (no overflow ✓).

Design meets spec with absolute minimum hardware (3 gates).

WARNING **Design traps in GCE exams**

- Forgetting inverters: every complemented literal in the SOP needs a NOT gate (unless input is available in both polarities).
- Fan-in limits: if the spec says "2-input gates only", a 3-input AND must be built from two 2-input ANDs. Count gates accordingly.
- Don't-cares: if the spec says "input combination X never occurs", put X in K-map and use it to enlarge groups.
- Verification: always test the simplified expression on the original truth table rows, not just the K-map.

DID YOU KNOW? **Integration Activity (Lesson 17) – Typical GCE Task**

A past GCE question asked: "A security system has sensors A (door), B (window), C (motion). Alarm Z sounds if door OR window opens while system armed ($S = 1$), OR if motion detected while armed. Design the minimal circuit." Approach: 1) Truth table (4 inputs S, A, B, C). 2) SOP. 3) 4-var K-map. 4) Circuit. 5) Verify. This integrates Lessons 15 & 16.

EXERCISE 6 **Integration: Security system design**

A security system has four inputs: S (system armed), D (door sensor), W (window sensor), M (motion sensor). The alarm A sounds ($A = 1$) when the system is armed ($S = 1$) AND (door open $D = 1$ OR window open $W = 1$ OR motion detected $M = 1$).

1. Write the truth table for $A(S, D, W, M)$.
2. Derive the canonical SOP expression.
3. Minimise A using a 4-variable K-map.
4. Draw the minimal circuit using 2-input gates only (AND, OR, NOT).
5. How many gates are needed?

SOLUTION **Security system design**

1. Truth table: $A = 1$ when $S = 1$ and $(D + W + M = 1)$. Rows 8–15 have $S = 1$. Among those, $A = 0$ only for $D = W = M = 0$ (row 8). So $A = 1$ for rows 9,10,11,12,13,14,15 ($\Sigma m(9, 10, 11, 12, 13, 14, 15)$). 2. Canonical SOP: 7 minterms of 4 literals each. 3. K-map minimisation: 4-var map (S, D rows; W, M cols). The seven 1s form a block of 8 minus the $S = 1, D = W = M = 0$ cell. Groups: - $S = 1$ row (8 cells) minus one → not rectangular. - Better: Group the 1s. Actually, $A = S \cdot (D + W + M)$. K-map confirms: One group of 4 for SD? No. $A = S(D + W + M) = SD + SW + SM$. Check K-map: SD covers rows where $S = 1, D = 1$ (4 cells: 12,13,14,15). SW covers $S = 1, W = 1$ (4 cells: 10,11,14,15). SM covers $S = 1, M = 1$ (4 cells: 9,11,13,15). Union covers all 7 minterms. Minimal SOP: $A = SD + SW + SM$. 4. Circuit (2-input gates only): - 3 AND gates: $G_1 = SD$, $G_2 = SW$, $G_3 = SM$. - 2 OR gates: $G_4 = G_1 + G_2$, $G_5 = G_4 + G_3$. - Total: 5 gates (3 AND, 2 OR). No NOT gates needed. 5. Gate count: 5 gates.

KEY POINTS **Chapter summary – CSC05**

- Boolean Algebra: 10 fundamental laws (Identity, Null, Idempotence, Complement, Involution, Commutativity, Associativity, Distributivity, Absorption, De Morgan). Duality principle.
- Standard Forms: Minterm (product, 1=uncomplemented), Maxterm (sum, 1=complemented). Canonical SOP (Σm), Canonical POS (ΠM). Conversion via truth table indices.
- Minimisation: Algebraic (laws, patterns, consensus) and Karnaugh Maps (Gray code, power-of-2 rectangles, wrapping, largest groups, fewest groups, don't-cares).
- Analysis: Label gates → global expression → simplify → truth table → interpretation.
- Design: Spec → Truth table → SOP → K-map → Circuit (NOT → AND → OR) → Verify.
- Exam skills: Name every law, draw K-maps with Gray code, count gates/literals, verify designs.

8 EXERCISES

8.1 I check my knowledge

EXERCISE 7 **MCQ: Boolean laws**

Which of the following expressions is equivalent to $A + \overline{A}B$?

1. $A + B$
2. $A \cdot B$
3. $\overline{A} + B$
4. $A \oplus B$

EXERCISE 8 **True/False with justification**

State whether each statement is true or false. Justify your answer using Boolean algebra laws.

1. $\overline{A + B} = \overline{A} \cdot \overline{B}$
2. $A \cdot (B + C) = A \cdot B + C$
3. $A + \overline{A} = 1$
4. $A \oplus B = \overline{A}B + A\overline{B}$

EXERCISE 9 **Definitions**

Define the following terms in the context of Boolean algebra:

1. Minterm
2. Maxterm
3. Canonical Sum of Products (SOP)
4. Don’t-care condition

EXERCISE 10 **Direct calculations**

Given $A = 1, B = 0, C = 1$, evaluate the following expressions:

1. $F_1 = A \cdot \overline{B} + C$
2. $F_2 = \overline{(A + B)} \cdot C$
3. $F_3 = A \oplus B \oplus C$
4. $F_4 = (A + B) \cdot (\overline{A} + C)$

8.2 I practise

EXERCISE 11 **Algebraic proof and truth table**

Prove algebraically that $A + \overline{A}B = A + B$. Then verify the identity by constructing a complete truth table.

EXERCISE 12 **Complement and SOP using De Morgan**

Using De Morgan's theorems, find the complement of $F = (A + \overline{B}C)(D + \overline{E})$. Express the complement as a Sum of Products (SOP).

EXERCISE 13 **Canonical forms and algebraic simplification**

Given the truth table below for a 3-variable function $F(A, B, C)$:

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

1. Write the canonical SOP and canonical POS expressions for F.
2. Simplify F algebraically to a minimal SOP form.

EXERCISE 14 **Karnaugh map minimisation and circuit**

Minimise the 4-variable function $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$ using a Karnaugh map. Draw the resulting logic circuit using AND, OR, and NOT gates.

8.3 I prepare for the GCE

EXERCISE 15 **BCD function with don't-cares – 12 marks**

A BCD-based function is defined as $F(A, B, C, D) = \sum m(1.3, 7.9) + d(10, 11, 12, 13, 14, 15)$, where d denotes don't-care conditions.

1. Plot the function on a 4-variable Karnaugh map, clearly indicating the don't-care cells.
2. Obtain a minimal SOP expression for F by exploiting the don't-care conditions.
3. Draw the logic circuit for the minimal expression using only NAND gates.
4. How many gates are saved compared to an implementation using the canonical SOP (without don't-cares)?

EXERCISE 16 **Circuit equivalence – 10 marks**

Two logic circuits are given by the following Boolean expressions:

$$F_1 = \overline{A}B + A\overline{B}C + ABC$$

$$F_2 = (A + B)(\overline{A} + \overline{B} + C)$$

1. Simplify F_1 and F_2 algebraically to their minimal SOP forms.
2. Show that the two circuits are equivalent by proving that their minimal forms are identical.
3. Draw the circuit for the minimal expression using only NOR gates.

EXERCISE 17 **Security light system – 15 marks**

A security light turns on if it is night AND (motion is detected OR the manual switch is on). Let the inputs be: N = night (1 if night, 0 otherwise), M = motion detected (1 if detected, 0 otherwise), S = manual switch (1 if on, 0 if off). The output L = light on (1 if on, 0 if off).

1. Write the Boolean expression for L in terms of N, M, S.
2. Construct the truth table for L.
3. Minimise the expression using a Karnaugh map.
4. Implement the minimised expression using only NAND gates. Draw the circuit.
5. If the manual switch is faulty and always reads 1 (on), simplify the expression for this faulty condition and state whether the light will ever turn off at night.

9 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: Boolean laws**

The expression $A + \overline{A}B$ can be simplified using the absorption law (also called the redundancy law):

$$A + \overline{A}B = (A + \overline{A})(A + B) = 1 \cdot (A + B) = A + B.$$

Alternatively, using distribution: $A + \overline{A}B = A(1 + B) + \overline{A}B = A + AB + \overline{A}B = A + B(A + \overline{A}) = A + B$.
Thus the equivalent expression is $A + B$, which corresponds to option 1.

SOLUTION **Exercise 2 — True/False with justification**

1. True. $\overline{A + B} = \overline{A} \cdot \overline{B}$ is De Morgan's first theorem.
2. False. $A \cdot (B + C) = A \cdot B + A \cdot C$ by the distributive law. The given expression $A \cdot B + C$ is missing the factor A on C. Counterexample: $A = 0, B = 0, C = 1$ gives LHS = 0, RHS = 1.
3. True. $A + \overline{A} = 1$ is the complement law.
4. True. $A \oplus B = \overline{A}B + A\overline{B}$ is the definition of the XOR (exclusive OR) operation.

SOLUTION **Exercise 3 — Definitions**

1. Minterm: A product term in which each of the n variables appears exactly once, either in its true or complemented form. For n variables there are 2^n minterms. A minterm evaluates to 1 for exactly one combination of the variables.
2. Maxterm: A sum term in which each of the n variables appears exactly once, either in its true or complemented form. For n variables there are 2^n maxterms. A maxterm evaluates to 0 for exactly one combination of the variables.
3. Canonical Sum of Products (SOP): A Boolean expression written as a sum (OR) of minterms. It is the logical OR of all minterms for which the function output is 1. Also called minterm expansion or standard SOP.
4. Don't-care condition: An input combination for which the output value is unspecified (can be either 0 or 1) because that combination never occurs in practice or its output does not affect the system. Don't-care conditions are denoted by d or X and can be used to simplify logic expressions further.

SOLUTION **Exercise 4 — Direct calculations**

Given $A = 1, B = 0, C = 1$:

1. $F_1 = A \cdot \overline{B} + C = 1 \cdot \overline{0} + 1 = 1 \cdot 1 + 1 = 1 + 1 = \mathbf{1}$.
2. $F_2 = \overline{(A + B)} \cdot C = \overline{(1 + 0)} \cdot 1 = \overline{1} \cdot 1 = 0 \cdot 1 = \mathbf{0}$.
3. $F_3 = A \oplus B \oplus C$. Compute stepwise: $A \oplus B = 1 \oplus 0 = 1$. Then $1 \oplus C = 1 \oplus 1 = \mathbf{0}$.
4. $F_4 = (A + B) \cdot (\overline{A} + C) = (1 + 0) \cdot (\overline{1} + 1) = 1 \cdot (0 + 1) = 1 \cdot 1 = \mathbf{1}$.

SOLUTION **Exercise 5 — Algebraic proof and truth table**

Algebraic proof:

$$\begin{aligned} A + \overline{A}B &= (A + \overline{A})(A + B) && \text{(Distributive law: } X + YZ = (X + Y)(X + Z)\text{)} \\ &= 1 \cdot (A + B) && \text{(Complement law: } A + \overline{A} = 1\text{)} \\ &= A + B && \text{(Identity law: } 1 \cdot X = X\text{).} \end{aligned}$$

Truth table verification:

A	B	$\overline{A}$	$\overline{A}B$	$A + \overline{A}B$	$A + B$
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	0	1	1
1	1	0	0	1	1

The columns for $A + \overline{A}B$ and $A + B$ are identical, confirming the identity.

SOLUTION **Exercise 6 — Complement and SOP using De Morgan**

Given $F = (A + \overline{B}C)(D + \overline{E})$.

The complement $\overline{F}$ is obtained by applying De Morgan's theorems:

$$\begin{aligned}
\overline{F} &= \overline{(A + \overline{B}C)(D + \overline{E})} \\
&= \overline{A + \overline{B}C} + \overline{D + \overline{E}} && \text{(De Morgan: } \overline{XY} = \overline{X} + \overline{Y}\text{)} \\
&= \overline{A} \cdot \overline{\overline{B}C} + \overline{D} \cdot \overline{\overline{E}} && \text{(De Morgan: } \overline{X + Y} = \overline{X}\,\overline{Y}\text{)} \\
&= \overline{A} \cdot (\overline{\overline{B}} + \overline{C}) + \overline{D} \cdot E && \text{(De Morgan on } \overline{\overline{B}C}\text{)} \\
&= \overline{A} \cdot (B + \overline{C}) + \overline{D}E && \text{(Double complement: } \overline{\overline{B}} = B\text{)} \\
&= \overline{A}B + \overline{A}\,\overline{C} + \overline{D}E.
\end{aligned}$$

The final expression $\overline{F} = \overline{A}B + \overline{A}\,\overline{C} + \overline{D}E$ is a Sum of Products (SOP).

SOLUTION **Exercise 7 — Canonical forms and algebraic simplification**

Truth table:

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

1. Canonical SOP (minterms where F=1): Rows 0, 2, 3, 5, 7 (minterm indices m_0, m_2, m_3, m_5, m_7).

   $$F = \overline{A}\,\overline{B}\,\overline{C} + \overline{A}B\,\overline{C} + \overline{A}BC + A\,\overline{B}C + ABC.$$

   Canonical POS (maxterms where F=0): Rows 1, 4, 6 (maxterms M_1, M_4, M_6).

   $$F = (A + B + \overline{C})(\overline{A} + B + C)(\overline{A} + \overline{B} + C).$$

2. Algebraic simplification to minimal SOP:

$$\begin{aligned}
F &= \overline{A}\,\overline{B}\,\overline{C} + \overline{A}B\,\overline{C} + \overline{A}BC + A\,\overline{B}C + ABC \\
&= \overline{A}\,\overline{C}(\overline{B} + B) + \overline{A}BC + AC(\overline{B} + B) && \text{(group terms)} \\
&= \overline{A}\,\overline{C} \cdot 1 + \overline{A}BC + AC \cdot 1 \\
&= \overline{A}\,\overline{C} + \overline{A}BC + AC.
\end{aligned}$$

Now factor $\overline{A}$ from the first two terms:

$$\overline{A}\,\overline{C} + \overline{A}BC = \overline{A}(\overline{C} + BC) = \overline{A}(\overline{C} + B) \quad \text{(by absorption: } \overline{C} + BC = \overline{C} + B\text{)}.$$

Thus

$$F = \overline{A}(\overline{C} + B) + AC = \overline{A}\,\overline{C} + \overline{A}B + AC.$$

This is a minimal SOP (3 product terms, 6 literals). No further reduction is possible.

SOLUTION **Exercise 8 — Karnaugh map minimisation and circuit**

The function is $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$.
Karnaugh map (4 variables):

Group 1: $\overline{B}\,\overline{D}$

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	1	1	0
10	1	0	0	1

Group 2: BD

K-map for $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$ with groupings

Grouping:

- Group 1 (blue): The four corners (m0, m2, m8, m10) → $\overline{B}\,\overline{D}$ (A and C change, B=0, D=0).
- Group 2 (orange): The middle four cells (m5, m7, m13, m15) → BD (A and C change, B=1, D=1).

No other groups needed. The minimal SOP is:

$$F = \overline{B}\,\overline{D} + BD.$$

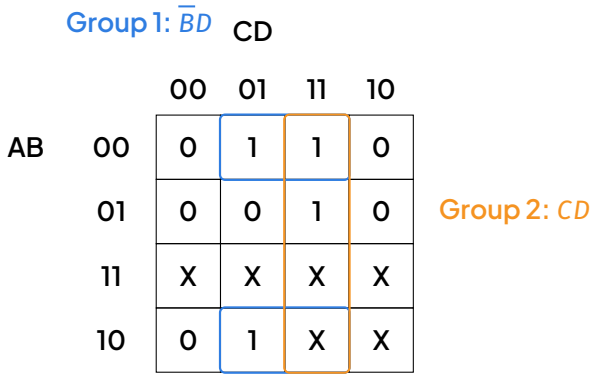
Logic circuit (AND, OR, NOT):

Logic circuit for $F = \overline{B}\,\overline{D} + BD$

SOLUTION **Exercise 9 — BCD function with don't-cares – 12 marks**

The function is $F(A, B, C, D) = \sum m(1.3, 7.9) + d(10, 11, 12, 13, 14, 15)$. (Note: the original notation "1.3,7.9" is interpreted as minterms 1, 3, 7, 9.)

1. Karnaugh map with don't-cares:



K-map for $F = \sum m(1.3, 7.9) + d(10, 11, 12, 13, 14, 15)$ with optimal groupings

2. Minimal SOP using don't-cares: We form the following prime implicants by treating X as 1 where helpful:

- Group 1 (blue): Minterms 1 (0001), 3 (0011), 9 (1001), 11 (1011) — using don't-care 11. These share B=0, D=1 → $\overline{B}D$.
- Group 2 (orange): Minterms 3 (0011), 7 (0111), 11 (1011), 15 (1111) — using don't-cares 11, 15. These share C=1, D=1 → CD.

All minterms (1, 3, 7, 9) are covered. The minimal SOP is:

$$F = \overline{B}D + CD.$$

3. Logic circuit using only NAND gates: We implement $F = \overline{B}D + CD$ using NAND-NAND logic (AND-OR equivalent to NAND-NAND).

$$F = \overline{\overline{\overline{B}D} \cdot \overline{CD}}.$$

Circuit:

- Generate $\overline{B}$ using a NAND gate as inverter (inputs B and B).
- First NAND: inputs $\overline{B}$ and D → output $\overline{\overline{B}D}$.
- Second NAND: inputs C and D → output $\overline{CD}$.
- Third NAND: inputs from first and second NAND outputs → final output F.

4. Gate count comparison:

- Canonical SOP (without don't-cares): Minterms 1,3,7,9. Each minterm is a 4-input AND (or 3 two-input AND gates per minterm). 4 minterms → 12 two-input AND gates + 3 two-input OR gates to combine + inverters for complemented literals. Roughly 15–20 gates.
- Minimal SOP with don't-cares: $F = \overline{B}D + CD$. Using NAND-only as above: 1 inverter (NAND as NOT) + 3 NAND gates = 4 NAND gates.
- Gates saved: Approximately 11–16 gates saved. The minimal implementation is drastically smaller.

Mark scheme (indicative):

- Correct K-map with don't-cares: 2 marks
- Correct minimal SOP: 3 marks
- Correct NAND-only circuit: 4 marks
- Gate count comparison with reasoning: 3 marks

Examiner expectations: Clear K-map labelling, correct grouping using don't-cares, minimal expression, proper NAND gate conversion (using De Morgan), and a realistic gate count estimate.

SOLUTION **Exercise 10 — Circuit equivalence – 10 marks**

Given:

$$F_1 = \overline{A}B + A\overline{B}C + ABC, \qquad F_2 = (A + B)(\overline{A} + \overline{B} + C).$$

1. Simplify F_1 to minimal SOP:

$$\begin{aligned} F_1 &= \overline{A}B + A\overline{B}C + ABC \\ &= \overline{A}B + AC(\overline{B} + B) \\ &= \overline{A}B + AC \cdot 1 \\ &= \overline{A}B + AC. \end{aligned}$$

2. Simplify F_2 to minimal SOP:

$$\begin{aligned} F_2 &= (A + B)(\overline{A} + \overline{B} + C) \\ &= A(\overline{A} + \overline{B} + C) + B(\overline{A} + \overline{B} + C) \quad \text{(Distributive)} \\ &= A\overline{A} + A\overline{B} + AC + B\overline{A} + B\overline{B} + BC \\ &= 0 + A\overline{B} + AC + \overline{A}B + 0 + BC \\ &= \overline{A}B + A\overline{B} + AC + BC. \end{aligned}$$

Now simplify using the consensus theorem or a truth table. The consensus of $A\overline{B}$ and BC is AC, and the consensus of $\overline{A}B$ and AC is BC. Both $A\overline{B}$ and BC are redundant when $\overline{A}B + AC$ is present. Verification by truth table:

A	B	C	$\overline{A}B + AC$	$A\overline{B} + \overline{A}B + AC + BC$	Equal?
0	0	0	0	0	Y
0	0	1	0	0	Y
0	1	0	1	1	Y
0	1	1	1	1	Y
1	0	0	0	0	Y
1	0	1	1	1	Y
1	1	0	0	0	Y
1	1	1	1	1	Y

Both reduce to the same minimal SOP:

$$F = \overline{A}B + AC.$$

3. Circuit using only NOR gates: We have $F = \overline{A}B + AC$. Using De Morgan:

$$F = \overline{\overline{\overline{A}B + AC}} = \overline{(\overline{\overline{A}B}) \cdot (\overline{AC})} = \overline{(A + \overline{B})(\overline{A} + \overline{C})}.$$

Implement $\overline{F} = (A + \overline{B})(\overline{A} + \overline{C})$ with NOR–NOR logic, then invert.

1. Generate $\overline{A}, \overline{B}, \overline{C}$ using NOR gates as inverters (inputs tied together).
2. NOR1: inputs $A, \overline{B} \rightarrow \overline{A + \overline{B}}$.
3. NOR2: inputs $\overline{A}, \overline{C} \rightarrow \overline{\overline{A} + \overline{C}}$.
4. NOR3: inputs from NOR1 and NOR2 $\rightarrow \overline{\overline{A + \overline{B}} + \overline{\overline{A} + \overline{C}}} = (A + \overline{B})(\overline{A} + \overline{C}) = \overline{F}$.
5. NOR4: inverter on NOR3 output $\rightarrow F$.

Total: 7 NOR gates (3 inverters + 4 logic).

NOR-only circuit for $F = \overline{A}B + AC$ (7 NOR gates)

Mark scheme (indicative):

- Simplification of F_1: 2 marks
- Simplification of F_2: 3 marks
- Proof of equivalence (showing both reduce to same minimal SOP): 2 marks
- Correct NOR-only circuit: 3 marks

Examiner expectations: Clear algebraic steps, recognition of consensus/redundancy, correct conversion to NOR logic (using double complement or POS form), neat circuit diagram with labelled gates.

SOLUTION **Exercise 11 — Security light system – 15 marks**

1. Boolean expression: The light turns on if it is night AND (motion detected OR manual switch on).

$$L = N \cdot (M + S).$$

2. Truth table:

N	M	S	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

3. Karnaugh map minimisation: K-map for 3 variables (N, M, S). Rows: N, Columns: MS (00.01,11.10).

		MS			
		00	01	11	10
N	0	0	0	0	0
	1	0	1	1	1

Group: $M + S$

K-map for security light system

Group the three 1's in the bottom row (N=1, MS=01,11,10) → $M+S$ (since only MS=00 is missing). So minimal expression:

$$L = N(M + S).$$

This is already minimal (same as original).

4. Implementation using only NAND gates: Convert to SOP: $L = NM + NS$. NAND-NAND implementation:

$$L = \overline{\overline{NM} \cdot \overline{NS}}.$$

1. NAND1: inputs N, M → $\overline{NM}$.
2. NAND2: inputs N, S → $\overline{NS}$.

3. NAND3: inputs from NAND1 and NAND2 → $\overline{\overline{NM} \cdot \overline{NS}} = NM + NS = N(M + S)$.

This uses 3 NAND gates (no separate inverters needed).
5. Faulty switch condition (S always 1): Substitute $S = 1$ into the minimal expression:

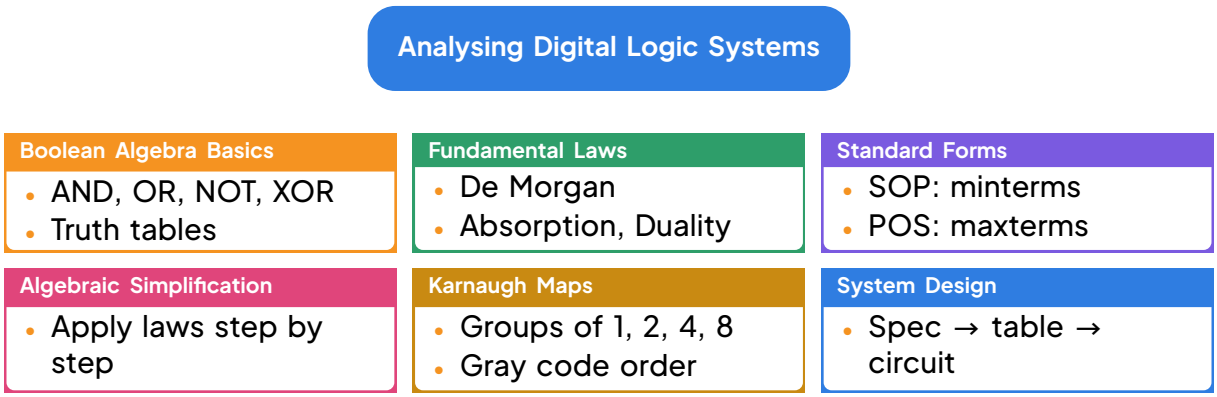
$$L = N(M + 1) = N \cdot 1 = N.$$

So the light turns on whenever it is night ($N = 1$), regardless of motion. The light will never turn off at night (since $L = N$, when $N = 1$, $L = 1$ always). During the day ($N = 0$), the light is off.
Mark scheme (indicative):

- Boolean expression: 2 marks
- Truth table: 2 marks
- K-map and minimisation: 3 marks
- NAND-only circuit: 4 marks
- Faulty switch analysis: 4 marks

Examiner expectations: Correct interpretation of the problem statement, complete truth table, proper K-map grouping, efficient NAND implementation (using SOP form), and clear analysis of the faulty condition with a logical conclusion.

10 SUMMARY SHEET



Mind map of the chapter: from Boolean laws to complete logic system design.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- A **Boolean variable** takes only two values: 0 (false, LOW) or 1 (true, HIGH). A **Boolean function** $F(A, B, C, \dots)$ associates an output in $\{0, 1\}$ to each combination of its input variables.
- **Boolean operations**: AND ($A \cdot B$, also written AB), OR ($A + B$), NOT ($\bar{A}$); derived gates: NAND ($\overline{A \cdot B}$), NOR ($\overline{A + B}$), XOR ($A \oplus B = \bar{A}B + A\bar{B}$, output 1 when the inputs differ), XNOR ($\overline{A \oplus B}$, output 1 when the inputs are equal).
- A **truth table** lists the output of a function for all 2^n input combinations of n variables (e.g. 8 rows for 3 variables, 16 rows for 4 variables).
- A **literal** is a variable or its complement (A or $\bar{A}$). A **minterm** is a product (AND) containing every variable exactly once, complemented or not; a **maxterm** is a sum (OR) containing every variable exactly once. With n variables there are 2^n minterms $m_0, \dots, m_{2^n-1}$ and 2^n maxterms $M_0, \dots, M_{2^n-1}$, numbered by the binary value of the row.
- **SOP** (Sum of Products): OR of AND terms, e.g. $F = \bar{A}B + AC$. **POS** (Product of Sums): AND of OR terms, e.g. $F = (A + B)(\bar{B} + C)$.
- **Canonical form**: every term contains all the variables. Canonical SOP is the OR of the minterms where $F = 1$, written $F = \Sigma m(\dots)$; canonical POS is the AND of the maxterms where $F = 0$, written $F = \Pi M(\dots)$.
- A **Karnaugh map** (K-map) is a graphical truth table whose rows and columns are labelled in **Gray code** so that two adjacent cells differ by exactly one variable.

- An **implicant** is a group of adjacent 1-cells; a **prime implicant** cannot be enlarged; an **essential prime implicant** covers at least one 1-cell that no other prime implicant covers.

Laws and theorems to know

- Identity: $A + 0 = A$, $A \cdot 1 = A$; Domination (annulment): $A + 1 = 1$, $A \cdot 0 = 0$.
- Idempotence: $A + A = A$, $A \cdot A = A$; Complement: $A + \bar{A} = 1$, $A \cdot \bar{A} = 0$; Involution (double negation): $\bar{\bar{A}} = A$.
- Commutative: $A + B = B + A$, $AB = BA$; Associative: $(A + B) + C = A + (B + C)$, $(AB)C = A(BC)$.
- Distributive: $A(B+C) = AB+AC$ and $A+BC = (A+B)(A+C)$ (the second form has no analogue in ordinary algebra — learn it).
- Absorption: $A + AB = A$; $A(A + B) = A$; and the very useful variant $A + \bar{A}B = A + B$ (provable by distributivity: $A + \bar{A}B = (A + \bar{A})(A + B) = 1 \cdot (A + B)$).
- Consensus: $AB + \bar{A}C + BC = AB + \bar{A}C$ (the term BC is redundant).
- **De Morgan's theorems**: $\overline{A + B} = \bar{A} \cdot \bar{B}$ and $\overline{A \cdot B} = \bar{A} + \bar{B}$, extendable to any number of variables: the complement of a sum is the product of the complements, and vice versa.
- **Duality principle**: in any valid identity, swapping every $+$ with $\cdot$ and every 0 with 1 gives another valid identity (e.g. $A + 0 = A$ dualises to $A \cdot 1 = A$).
- Complement of a function: $\bar{F}$ is given by the minterms where $F = 0$; equivalently $F = \Sigma m(\text{1-cells}) = \Pi M(\text{0-cells})$, and the indices satisfy $\Sigma m(\text{list}) = \Pi M(\text{complementary list})$.
- NAND and NOR are **universal gates**: any Boolean function can be built from NAND gates alone (or NOR gates alone), using De Morgan to convert SOP into NAND-NAND form.

Methods

1. Algebraic simplification: expand products if needed, factor common literals, then apply absorption, the complement law ($X + \bar{X} = 1$), idempotence (a term may be duplicated: $A = A + A$) and De Morgan. Justify every step by naming the law used.
2. Truth table → canonical SOP: for each row where $F = 1$, write the minterm (a variable appears complemented if its value is 0 in that row, uncomplemented if it is 1), then OR all the minterms. For canonical POS, take the rows where $F = 0$ and write the maxterm of each (a variable appears complemented if its value is 1), then AND them.
3. K-map minimisation (SOP): (a) draw the map with Gray-coded labels 00, 01, 11, 10; (b) plot the 1s (plot the 0s instead to obtain a POS result); (c) group the 1s into rectangles of 1, 2, 4, 8 cells (powers of 2 only), as large as possible, remembering that opposite edges are adjacent (wrap-around); (d) cover every 1, selecting all essential prime implicants first; (e) each group yields one product term containing only the literals whose value does not change inside the group; (f) OR the terms.

4. Designing a logic system: read the specification → identify the inputs and outputs → build the truth table → derive the canonical expression → minimise it (algebraically or by K-map) → draw the circuit with logic gates → verify the circuit against the truth table for a few input combinations.

Worked micro-example (verification habit)

- Simplify $F = \bar{A}B + AB$. By distributivity $F = B(\bar{A} + A)$; by complement $F = B \cdot 1 = B$. Check with the table: $F = 1$ exactly when $B = 1$ (rows $AB = 01$ and 11), so $F = B$ is confirmed.

Common mistakes

- Writing $\overline{A + B} = \bar{A} + \bar{B}$: De Morgan changes the operator — the correct form is $\overline{A + B} = \bar{A} \cdot \bar{B}$.
- Ordering K-map columns as 00, 01, 10, 11 (binary counting) instead of the Gray code order 00, 01, 11, 10; with the wrong order, adjacency is lost and groupings become invalid.
- Forgetting that K-map edges wrap around: the first and last columns (and rows) are adjacent, so e.g. cells in columns 00 and 10 can be grouped.
- Making groups of 3, 5 or 6 cells: only powers of 2 (1, 2, 4, 8, 16) are allowed.
- Confusing minterm and maxterm conventions: for a minterm, complement the variable when its value is 0; for a maxterm, complement it when its value is 1.
- Reading a K-map group term with a variable that changes value inside the group: such a variable must be eliminated, not kept.
- Leaving an expression "simplified" while it still contains a redundant consensus term or a term absorbable by $A + AB = A$.
- Mixing up XOR and OR: $A \oplus B = 1$ requires the inputs to differ, whereas $A + B = 1$ also holds when both are 1.

GCE tips

- Always show each simplification step with the law named beside it: method marks are awarded even if the final answer slips.
- Check a simplified result by testing one or two input combinations against the truth table — a one-minute check that catches most errors.
- In K-map questions, draw the map neatly with Gray-coded labels, circle the groups clearly, and state the product term corresponding to each group before writing the final sum.
- If asked for the cheapest or fastest circuit, count gates and gate inputs: minimisation reduces both the component cost and the propagation delay.
- When the question imposes NAND-only (or NOR-only) implementation, take the minimal SOP and apply De Morgan twice rather than redesigning from scratch.

- Manage time: a 4-variable K-map should take under five minutes with practice; memorise the Gray code order 00, 01, 11, 10 so you never hesitate.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+